The
LHASA Transform Writers' Guide

Reformatted for Microsoft Word

July 2017

Minor revisions

August 2017

Acknowledgements

The first edition of this document was written by Anthony Baxter (ICI Organics) and Alan Long (Harvard University). Since that edition, a final section consisting of a worked transform-writing example has been added by Alan Long, the section on pattern-keyed transforms has been rewritten by Paul Hoyle (University of Leeds), and a new section on ratings has been added at Harvard.

Several other people have made suggestions, comments, and additions that were helpful to us, among them:

Susan Bostock (ICI),
Glen Hopkinson (Leeds),
Peter Johnson (Leeds),
Philip Judson (Schering Agrochemicals PLC),
Chris Marshall (Leeds),
Stewart Rubenstein (Harvard), and
Harkamal Tumber (Leeds).

Notes on this reformatted edition

The original Writers' Guide was created using the runoff editor which is now obsolete. The current edition has been created by reformatting the file as a Word document by Philip Judson. In as far as possible, the original layout has been retained. The original text has not been altered except as follows.

It was difficult to work out header numbering, which would have been generated automatically by runoff, reliably. There are only a few references to header numbers in the text and so these have been replaced with references to the header names themselves. Appendix VI has been added, describing the recently-devised NEW*1D*PATTERNs.

The runoff file contained placeholders for figures and diagrams. They were not included in the body of the file. At the time of reformatting, the associated image files were not available. In some cases it was clear what the figure or diagram would have been and it has been recreated. In other cases this was not possible and there are several instances of the same figure being referenced in two placeholder positions even though the related text would seem to require the two figures to be different. Where it was not clear what the image should be, the placeholders have been retained in angle brackets. Many extracts from transforms, being in CHMTRN, include diagrams created from the keyboard and these use alignment of letters and spaces that depend on the choice of font and font size. They are likely to be rendered incorrectly if the choice of font for the document is changed from the one used for its creation – Times New Roman, point size 12.

The conversion from runoff to Word has been done to extend the life of the Writers' Guide.

Philip Judson
24th July 2017

Contents

Introduction

This CHMTRN Writers' Guide is intended for use by chemists who are already familiar with the operation of the LHASA program for computer-assisted synthetic analysis. For those not yet trained in the use of the program, it is recommended that some time be spent in familiarization, both through reading the LHASA User's Guide and through actually running the program. The writer will also have to become familiar with the VAX/VMS operating system and an available text editor.

This guide is intended to assist chemists interested in expanding the chemical knowledge base for the LHASA program. The knowledge base is written entirely in the chemical English language, CHMTRN (for CHeMistry TRaNslator). An exhaustively, and occasionally exhaustingly, detailed dictionary to the CHMTRN language is available, the LHASA CHMTRN Manual.

Since the CHMTRN Manual may slip out of date from time to time, the reader should know that the real last word on the language is contained in two files, CHMTRN.WRD and CHMTRN.GRM, contained in the disk directory [LHASAxx.AUX.CHMTRN].

When written documentation on CHMTRN becomes too dry and/or incomprehensible, it is often instructive to look at existing CHMTRN knowledge-base tables for actual examples of the use of a particular capability. The VAX/VMS SEARCH command is very useful in this regard.

What Do You Put into a Transform?

We cannot emphasize too strongly that LHASA is not an information retrieval program designed to regurgitate specific examples of known reactions. The knowledge base of transforms in LHASA has been designed to allow LHASA to look at an arbitrary input target structure, and to decide, by asking intelligent structural questions, which transforms are appropriate and which are not. Accordingly, the task of the transform writer extends far beyond coding the mechanical transformations of A to B, C to D, etc. As a more concrete example, it is obvious that LHASA can make intelligent decisions about a broader range of target structures when it asks a question like:

If location X is better able to bear a positive charge than locationY then....

than if it poses only specific queries like:-

If there is a bromide on location X then... but if there is a hydroxyl then....

As you will see from reading the section, “What is CHMTRN”, and the CHMTRN Manual referred to above, it is possible to ask both of these types of questions in LHASA's CHMTRN language. We can only entreat you, as future CHMTRN coders, to remember this when adding to the LHASA knowledge base:

THINK BROAD!

For example, if a reaction is reported in the literature as occurring only for a few withdrawing groups, e.g. ester and cyano, and no reference can be found to it not working for the others, e.g. nitro, then consider whether you should include them. It may well be that no one has tried the others because they were not interested in them at that time. Use your chemical intuition to decide what would work and what wouldn't.

Before a transform is written, a comprehensive literature survey of the reaction or reaction class should be conducted. It is important to encompass both reactions that have worked and those that have not worked. As much information as possible should be collected, e.g. yields, effect of ubstituents, etc.

Once the information is gathered, it should be systematized so that generalizations can be made. From these generalizations it should be possible to devise an overall plan for the transform.

Who Writes a Transform?

There are of course a large number of possibilities for dividing up the responsibilities involved in writing a transform. We cite here two systems that are currently in use and invite you to elaborate, refine, and/or revolutionize as appropriate.

At ICI Organics in England, writing a transform is a collaborative venture between a "volunteer" chemist and a "coder" chemist. The volunteer, who is preferably already quite familiar with the area of chemistry in question, does a thorough literature search, abstracts the assembled information onto a "LHASA Reaction Summary Form" (Appendix III) and then passes the form on to the "coder", another chemist who actually writes the transform entry. Note that the sample summary form in Appendix III is not meant to be restrictive. Frequently, extra sheets are appended - photocopies of tables of yields and substituents, for example, are a very useful basis from which to extrapolate.

The coder must decide, in light of his or her chemical knowledge, whether sufficient data have been provided. If not, more is requested. After coding the transform, the coder tests the new entry with examples supplied on the coding form and with any others deemed appropriate. Finally, the "volunteer" tests the transform to see if it is satisfactory. At all stages there is a dialog between the "coder" and the "volunteer" to ensure the best possible result.

At Eli Lilly in Indiana, a system has been devised for collaborative transform writing. A "volunteer" does the literature search and presents his or her findings to a committee, which pools its CHMTRN knowledge for the construction of the actual transform entry. Testing and debugging is then completed by the volunteer, with help from colleagues as necessary.

Neither of these systems is perhaps optimal, but both have produced effective transforms for the LHASA knowledge base. The ICI system is more appropriate when an organization has a resident CHMTRN expert, and the Lilly system good when several people can be found who enjoy the challenge of trying to talk chemistry with a computer.

What is CHMTRN?

CHMTRN language

CHMTRN is the language in which the knowledge-base tables for LHASA are written. It is a chemical English language that allows the chemist to ask relevant questions about a target structure and to perform various operations in LHASA depending on the answers. CHMTRN is made up of "qualifiers", or sentences, for example:

```
ADD 15 IF THERE IS AN ESTER ON ATOM*1.
```

This typical qualifier has a "target", or structural feature (ESTER), a "locant", or position in the molecule (ATOM*1), and an "operation" (ADD 15). The rest of the words in this particular qualifier are "buzz words", which have no meaning to LHASA and are included simply to enhance readability.

The CHMTRN language is made up, for the most part, of words, or "specifiers". These specifiers fall into a number of categories, some 15 in all, but the most common ones are targets, locants, and operations.

Targets are features of chemical structure: atom types, bond types, functional groups, rings, stereochemical features and relationships, etc. For example, HETERO, CARBON, CYANO, RING OF SIZE 6, AXIAL, and FUSION are all target specifiers in CHMTRN.

Locants are locations within a chemical structure, atoms and bonds along a "path" of atoms (e.g. ATOM*1, BOND*3), or atoms and bonds in functional groups (CARBON1*2, HETERO1*2, BOND1*1, etc. for GROUP*1). Locants may also be qualified by a distance specifier (ALPHA, BETA, or GAMMA) and by one or more additional specifiers (e.g. ONRING, OFFPATH, ON*THE*BRIDGE). A typical locant in a CHMTRN qualifier might then be:

...ALPHA TO ATOM*3 OFFPATH ONRING....

The most common operations in CHMTRN are ADD, SUBTRACT, and KILL. The first two of these modify the rating for a transform, and KILL completely eliminates a transform from consideration. In addition to these three basic operations, there are a number of "special operations" which do many things, from specifying CONDITIONS, LOOPing on variable values, requesting a functional group interchange subgoal (EXCHANGE), to sending a MESSAGE or WARNING to the chemist.

Complete lists of the available target, locant, and operation specifiers, along with scopes and limitations for their use, can be found in the CHMTRN Manual mentioned above. It is our intention in this Transform Writers' Guide to give you a sense of the kinds of things that can be done in CHMTRN and to avoid getting bogged down in the gory details.

How is CHMTRN Translated by LHASA?

Since online interpretation (reading) of the actual text of a CHMTRN knowledge-base table would be a very slow process, such tables are preprocessed by an offline program (also called CHMTRN). CHMTRN uses a definitions file, or dictionary, called CHMTRN.WRD, to assign a numerical value to each word in a CHMTRN qualifier and then packs these numerical values into a computer word. The various specifiers in a certain category are given unique values for that category. For example, targets are assigned values 0-511 in a specific "field" in the computer word, the QLTARG field. Similarly, there are fields for locants and fields for operations. In our example above, for instance, four fields in the computer word would be filled, with values QLOPER 1 (ADD), QLINCR 15 (15), QLTARG 4 (ESTER), and QLFROM 1 (ATOM*1). The other three most commonly used fields are QLDIST (=1,2,3 for ALPHA, BETA, GAMMA), QLNOT (=1 for NOT), and QLQUAL (used for qualified locant specifiers like ONRING, OFFPATH, etc.).

The above discussion may seem needlessly technical for the uninitiated reader. For the most part, translation of tables by CHMTRN and online interpretation of the packed, binary versions of these tables by LHASA will be completely transparent to the table writer. One very important lesson to be learned, however, is that it is strictly illegal to write a CHMTRN qualifier that would require more than one target specifier, more than one locant, or more than one operation to be packed into a single computer word. Fortunately, the CHMTRN program is very good at pointing out these types of transgressions.

To allow the CHMTRN writer to avoid mistakes of this sort, CHMTRN is supplied with a number of "word separators", specifiers that tell the CHMTRN program to put everything that follows into a new computer word and to turn on the QLCONT bit (a one-bit field) telling LHASA to read two words as one qualifier. The most common word separators are AND and THEN, though there are many others. As an example, consider the following pair of qualifiers:

KILL IF THE FRAGMENT*COUNT IS ONE
ADD 25 IF THE FRAGMENT*COUNT IS ONE

On the surface, these qualifiers both sound fine. The first one is OK – it has an operation (KILL), a QLINCR (ONE), and a target (FRAGMENT*COUNT) that stands alone and doesn't need a locant. The second qualifier has a problem, however. Both 25 and ONE are defined in CHMTRN as QLINCR's and the CHMTRN program cannot put them into the same field. The proper way around this difficulty is to force one of the QLINCR's into the next computer word by using a word separator, e.g.

IF THE FRAGMENT*COUNT IS ONE THEN ADD 25.

It is also important to get a little insight into the way that LHASA evaluates qualifiers. In the simplest case, the program makes up a set of atoms (or bonds) in the molecule that have the target characteristics and another set of atoms (or bonds) that satisfy the locant criteria. LHASA then logically AND's these sets together and looks at the resultant set to determine whether the qualifier is true or false. If the resultant set is non-zero, the qualifier is true and the requested operation is performed. If the resultant set is empty, the qualifier is false and the operation is skipped and the next qualifier unpacked. In our first example,

ADD 15 IF THERE IS AN ESTER ON ATOM*1,

the target set would be filled with the carbon origins of all the ESTER groups in the molecule. The locant set would be filled with the single atom labeled as ATOM*1 of the path (it is possible to have multiple atoms in the locant set in the case of a more complicated locant, e.g. ALPHA TO ATOM*1 OFFPATH). If the atom labeled ATOM*1 appears in the set of atoms bearing ESTER groups, the resultant set will contain that atom and 15 will be added to the current transform rating. If not, no rating modification will occur.

While the majority of transform qualifiers are simple, like the one above, it is also possible to do quite sophisticated things in CHMTRN, which has evolved over the years into a programming language in its own right. A number of these more specialized capabilities are described in the section, “More Features of CHMTRN”.

What Does a Transform Look Like?

This section describes the general transform format. For the formats of each specific table, which may differ slightly from one to the next, the reader is referred to the section, “Specific Table and Transform Formats”. The style of a transform entry, i.e. the spacing, indentation, etc. is considered in the section, “Transform Writing Elements of Style”. In the generic transform entry below, note that any line preceded by 3 dots (not 4 dots) is a comment line as far as the CHMTRN program is concerned.)

TRANSFORM number (Note 1)
NAME of transform (Note 2)
...References (Note 3)
...Descriptive Picture (optional) (Note 4)
...1-D Retron Pattern (pattern-keyed TFs) (Note 5)
Initial Rating Specifiers (Note 6)

...2-D Retron and Precursor Patterns (Note 7)
...Transform Authorship (optional) (Note 8)
...Path Length/ Change (group-keyed TFs) (Note 9)
Keying Group Info. (group-keyed TFs) (Note 10)
Special Sets (Note 11)
BROKEN*BONDS (Note 12)
Stereochemical Sets (Note 13)
... (Note 14)
Prescreening Qualifiers (optional) (Note 15)
Target Qualifiers (Note 16)
.... (Note 17)
Mechanism Commands (Note 18)
.... (Note 19)
Precursor Qualifiers (Note 20)
CONDITIONS Statements (Note 21)
(Note 22)

Notes

1 The transform number is a unique number that must always appear first in any transform, as in

TRANSFORM 532.

The currently reserved transform numbers are:

1-399	PAIRTx	(two-group goal transforms)
400-449	OLFTB	(stereoselective C=C transforms)
450-499	RFGTB	(reactive functional group transforms)
500-699	SGTABx	(one-group goal transforms)
700-749	FGATBx	(functional group additions)
750-949	FGITBx	(functional group interchanges)
95 -969	FGRTB	(functional group removals)
970-999	TAUTOMER	(tautomerizations)
1000-2999	PCHEMx	(1-D pattern goal transforms)
3000-4999		(warnings, messages, and titles)
5000 and up	TCx	(tactical combinations)

2 The name of a transform does not have to be unique, but should be as descriptive as possible (in 63 or fewer characters) of the (synthetic) reaction, e.g.

NAME SN2 Epoxide Opening by Organometallic.

3 If possible, general references, e.g. text books or reviews, should be given. These references should enable the user to find more information about the reaction easily. If the list of references is extremely long, it should be included as a separate file, with a note in the transform about how to find it.

4 The descriptive picture of a transform is used mostly in pattern-chemistry transforms whose keying substructures (e.g. rings) are not obvious from the 1-D or 2-D patterns.

Like the references, all picture lines start with ... and are ignored by the CHMTRN compiler. An example is shown in the section, “Coding and Debugging”.

5 A growing proportion of the transform knowledge base is keyed by 1-D patterns that are parsed by an offline program (PATRAN) separate from the CHMTRN compiler. The 1-D pattern lines are preceded by ... and thus ignored by CHMTRN. 1-D patterns are enclosed by the special keywords STARTP and ENDP:

...STARTP
...keying pattern
...ENDP

An example of a 1-D pattern is shown in the section. “Coding and Debugging”, and the section, “Writing 1-D Patterns for LHASA”, is devoted to the rules for 1-D pattern writing.

6 There are two kinds of "ratings" for LHASA transforms. The "initial rating" is calculated from a weighted assessment of the utility of the corresponding reaction as measured according to several criteria, detailed in the section, “How is a Transform Rated?”. The initial rating is used for ordering possible multi-step sequences before they are performed. The decision as to whether to display a transform or not is made according to the value of the "final rating". The final rating is calculated by adding the initial rating and the rating "increment", the sum of the ADD, SUBTRACT, INCREMENT, and DECREMENT qualifiers in the transform. The final rating can also be changed by functional group interference identified by the program in the precursor(s). A transform is killed if its final rating is 0 or below.

7 A 2-D pattern for the target substructure ("or retron") and the precursor substructure should be included here as a comment. The 2-D patterns are used now for keying tactical combinations of transforms, for selecting transforms from the knowledge base to be used as subgoals for a tactical combination, and for checking the group-oriented transform keying in certain transform tables. 2-D patterns will be used more extensively in the future. Several examples of 2-D patterns are included in the sample transform headers in the section, “Individual Transform Tables”, and the section, “Writing 2-D Patterns for LHASA”, is devoted to the rules for 2-D pattern writing.

8. Transform authorship: the writer of a transform is asked to include name, affiliation, and approximate date. Major revisions to a transform should be identified by author in a similar fashion:

...Tony Baxter, ICI Organics, July 1982.
...Alan Long, Harvard, October 1984.

9 Many of the transform tables contain only transforms of a single PATH, which can be either the distance in bonds between two keying group carbon origins (in the PAIRTx tables), the distance in bonds from the keying group to the broken bond (in the SGTABx tables), or the distance that the functional group origin is moved (in subgoal tables FGITBx and FGATBx). The number at the end of the name (e.g. "x" in PAIRTx) determines the PATH for the purposes of the LHASA executives, and the ...PATH etc. comment line (see, for example, the subsections, “PAIRTx” and

"SGTABx") in a transform entry is included simply as a convenience to the person looking at the table.

In transforms keyed by 1-D patterns, the number of atoms in the pattern determines the path length.

Pattern-keyed transforms are not grouped according to path length or path change, but rather according to the substructure type(s) covered by the patterns. For example, all mono-hetero 5-membered-ring aromatic transforms might be grouped in the same PCHEMx table, even though they did not necessarily all have the same path length.

10 Keying group information is necessary in all the group-oriented knowledge-base tables: PAIRTx, SGTABx, FGITBx, FGATBx, RFGTB, and FGRTB. In PAIRTx, two groups must be specified, e.g.

GROUP*1 IS ALDEHYDE
GROUP*2 IS ALCOHOL

In SGTABx, RFGTB, FGATBx, and FGRTB, one group is specified:

GROUP*1 IS ESTER.

Note that in FGATBx tables, the group specified is not the keying group, but rather the group type being requested for the subgoal.

In the FGITBx tables, two groups are again specified, the keying group and the requested group, respectively, e.g.

SUBJECT GROUP IS ALDEHYDE OR KETONE
OBJECT GROUP IS ALCOHOL

for an alcohol oxidation functional group interchange.

11 Special Sets (optional) are used to speed up execution of the program by limiting the number of transforms attempted in a given processing mode. For example, the DISCONNECTIVE tactic on the processing menu will only try transforms containing the DISCONNECTIVE special set label.

12 Inclusion of BROKEN*BONDS specifiers (optional), as in

BROKEN*BONDS BOND*1 BOND2*2

allows LHASA to cross-reference transforms according to the bonds they break. This cross-referencing is useful both for speeding up strategic-bond-mode processing and for skipping transforms that break bonds designated by the chemist as PRESERVED.

13 The six stereochemical specifiers (REMOVES*STEREO, CREATES*STEREO, INVERTS*STEREO, RETAINS*STEREO, RACEMIZES*STEREO and REQUIRES*STEREO) work in much the same fashion as the BROKEN*BONDS sets, in this case cross-referencing transforms according to stereochemical outcome.

The *STEREO sets differ from the BROKEN*BONDS sets in that their arguments are atoms instead of bonds:

REMOVES*STEREO ATOM*1.

14 This blank comment line signifies the start of the CHMTRN qualifiers.

15 Qualifiers appearing before an (optional) END*PRESCREEN line are typically KILLs for situations that are not rectifiable by subgoals. Thus, these qualifiers should not refer to locants in the keying functional groups. No (further) subgoals will be attempted in pursuit of a transform that fails due to one of these prescreening qualifiers. After the transform evaluation has passed the END*PRESCREEN statement, or if there is no such statement, the transform is considered to be a reasonable candidate for subgoal requests.

16 These are the qualifiers that apply to the target molecule. Target qualifiers may be further subdivided into sections for PRESCREENing as discussed later in this text.

17 The first occurrence of the four dots indicates the start of the mechanism commands.

18 Mechanism commands are those CHMTRN qualifiers that perform the atom and bond manipulations necessary to convert target to precursor(s).

19 This second occurrence of four dots directs the program to perform the mechanism and generate and perceive the precursor(s). If there are no post-mechanism qualifiers, the precursor is displayed immediately. If there are post-mechanism qualifiers, display is deferred until after these have been read and evaluated.

20 These are the qualifiers that apply to the precursor molecule.

21 CONDITIONS statements may be placed anywhere in a transform entry after the start of the qualifiers. However, if there are no post-mechanism qualifiers, it is more efficient to place the CONDITIONS statement(s) somewhere before the second set of four dots.

If there are post-mechanism qualifiers, the CONDITIONS statement(s) should be placed immediately after the second set of four dots for consistency.

Actual conditions may now be specified for each CONDITIONS statement, e.g.

CONDITIONS pH<1 AND STRONGLY*ELECTROPHILIC
ACTUAL*CONDITIONS HNO3 in H2SO4

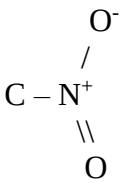
for an electrophilic aromatic nitration. If ACTUAL*CONDITIONS are specified for the preferred set of prototype CONDITIONS selected by LHASA based on minimum interference from non-participating functional groups, they will be shown to the chemist in place of the "Prototype conditions:" display.

22 Three blank lines should be left after each transform entry.

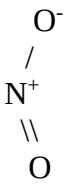
More Features of the CHMTRN Language

Target Specifiers

Usually targets in CHMTRN are fairly easily understood. Among the most important targets are the functional group types and the explicit atom types. It is important to remember that a LHASA functional group always resides on a carbon atom and includes that carbon. Thus, a NITRO group is:



and not



Functional groups should be described as being ON a locant, as in:

ADD 25 IF THERE IS A NITRO GROUP ON ATOM*1.

Note that while the functional groups are carbon-based, the explicit atom types are not. Thus both of the following qualifiers would be "true" for a target structure bearing -SR on ATOM*3 of the path:

ADD 15 IF THERE IS A SULFIDE ON ATOM*3
ADD 15 IF ALPHA TO ATOM*3 IS A SULFUR ATOM.

The CHMTRN targets FLUORINE, CHLORINE, BROMINE, and IODINE are not synonymous with FLUORIDE, CHLORIDE, BROMIDE, and IODIDE, and the correct one must be chosen for the particular application, one of the first group for atom types and one of the second for functional groups.

Note that targets HYDROGEN, ALKYL, and METHYL refer not to the substituents themselves, but to the atoms bearing those substituents, as in

SUBTRACT 5 IF THERE IS A HYDROGEN ON ATOM*4.

(The HYDROGEN(S) here can be either implicit or explicit.)

To refer to actual substituent atoms, the target specifiers HYDROGEN*ATOM, ALKYL*ATOM, and METHYL*ATOM can be used, as in

IF ALPHA TO ATOM*7 IS A METHYL*ATOM THEN....

Both explicit atom types and functional group types can be used in mechanism commands. For example, the following lines would convert a ketone to a thiocarbonyl:

```
REMOVE THE KETONE ON ATOM*3
ATTACH A SULFUR TO ATOM*3
DOUBLE THE NEWEST*BOND.
```

Locant specifiers

Locants are somewhat less chemical than targets and hence require a bit more effort to understand. There are four basic types of locants in CHMTRN: path atoms(bonds), group atoms(bonds), saved atoms(bonds), and specified atoms(bonds). All of these are important to know about, as they show up quite frequently.

The most common type of path is a chain of carbon atoms grown out from a functional group or strategic bond, or from one functional group to another. Atoms and bonds along the path are given labels so that they can be referred to in the CHMTRN code. For example, for an aldol transform, the keying substructure might be a carbonyl and a hydroxyl joined by a path of two bonds, labeled as follows:

<.figure 10>

In a substructure with two functional groups like this, the transform writer defines one as GROUP*1 and the other as GROUP*2. The "path" is then a continuous chain of atoms (and bonds) from the carbon origin of GROUP*1 to the carbon origin of GROUP*2, as shown above (assuming the carbonyl to be GROUP*1).

It is also possible to refer to atoms and bonds in the keying functional group(s) by specific names. In the simple example here, each functional group has one carbon and one hetero atom (hydrogens don't get labels) and only one bond, so the corresponding locant specifiers would be:

<.figure 10>

The entire labeling scheme for functional groups is explained in the CHMTRN Manual.

It is frequently the case that atoms and bonds not in the keying groups or on the CHMTRN path must be referred to by the transform writer. For example, the first carbon in the R group in the above example is the atom ALPHA TO ATOM*1 OFFPATH (OFFPATH excludes keying group atoms as well as path atoms). In cases like this, it is often convenient to assign a new name to the complex locant. Here, for example, one could say

```
SAVE AS 1 THE ATOM ALPHA TO ATOM*1 OFFPATH
```

and from that point on refer to the locant as SAVED*ATOM 1 rather than THE ATOM ALPHA TO ATOM*1 OFFPATH.

Sometimes, it is necessary to ask the same questions about two or more locants, and this is where SPECIFIED atoms and bonds are useful. For example, one might say something like

```
FOR EACH ATOM ALPHA TO CARBON1*1 DO ELIM*CK
  SUBTRACT 40 IF SPECIFIED*ATOM 1 IS ENOLIZABLE  &
    AND:IF THERE IS A LEAVING GROUP ON ALPHA  &
    TO SPECIFIED*ATOM 1 OFFPATH
ELIM*CK ENDDO
```

DO blocks and concatenated qualifiers are discussed in more detail later in this section. At the moment, it is only important to see that the SUBTRACT qualifier is read twice, once with the carbon in the R group as SPECIFIED*ATOM 1 and once with ATOM*2 as SPECIFIED*ATOM 1.

Conditional Execution

Most qualifiers in LHASA are conditional on the truth or falsehood of some sort of test. For the simple qualifier

```
ADD 15 IF THERE IS AN ESTER ON ATOM*1,
```

the operation ADD 15 is only performed if there **is** an ESTER ON ATOM*1.

It is also possible to concatenate qualifiers so that several operations can be performed in a single target/locant comparison is true, e.g.

```
IF THERE ARE FEWER THAN TWO HYDROGENS ON ATOM*1  &
  THEN TURN ON FLAG FEW*HYD AND*THEN ADD 15.
```

To make several qualifiers conditional on the truth of a particular qualifier, it is usually best to use a "block if" construction, e.g.

```
IF BOND*1 IS IN A RING
BEGIN RINGBLOCK
  ADD 25 IF BOND*1 IS IN A RING OF SIZE 5  &
    THEN EXIT RINGBLOCK
  KILL
BLKEND RINGBLOCK
```

The block of qualifiers from BEGIN to BLKEND is only executed if BOND*1 IS IN A RING is true. If BOND*1 is not a ringbond, control is passed to the location after the BLKEND RINGBLOCK statement. If the qualifier containing the EXIT is true, then the block is exited and control also passes to the location following the BLKEND.

Transfer of Control to Different Parts of the Table

As mentioned in the previous section, an EXIT in a conditional block of qualifiers, or the falsehood of the qualifier on which entrance into that block is based, both result in the transfer of control to a new location in the table, not just to the one immediately following the current location. There are a number of other ways of transferring control from one location to another in CHMTRN. By far the most common of these is the GO TO. A GO TO is normally used as the final part of a conditional qualifier, e.g.

```
IF THERE IS A KETONE ON ATOM*1 THEN GO TO KET1.
```

If the first qualifier is true, then control passes to the location of the address label KET1.

Another useful control statement, JUMP, is used mostly in the long-range search tables. JUMP is basically a computed GO TO, e.g.

```
JUMP ON (VAR1) TO LOCATIONS LABEL1 LABEL2 LABEL3
```

where VAR1 is a variable (all variables are integer in CHMTRN). Control transfers to LABEL1 if VAR1 is 1, to LABEL2 if VAR1 is 2, etc.

It is often necessary in a CHMTRN table to execute quite a large block of qualifiers for a specific purpose. In fact, it may be desirable to execute such a block for each of several different transforms. For example, a table may have several transforms that need to test for steric hindrance to nucleophilic displacement at a particular locant. In order to avoid having several copies of the same block of qualifiers, it is possible to code that block as a subroutine. CHMTRN subroutines may be called with up to 5 atoms and 5 bonds as arguments, as in

```
        CALL CHECKFG AT ATOM*1 AND ATOM*2.
```

Another place in the table might call CHECKFG with different arguments, e.g.

```
        CALL CHECKFG AT SAVED*ATOM 3 AND ATOM*4.
```

Within the subroutine CHECKFG, the first atom argument would be referred to as SPECIFIED*ATOM 1 and the second as SPECIFIED*ATOM 2. Bonds are handled in an analogous fashion. A subroutine may also be called with no arguments. The current path atom and bond labels, e.g. ATOM*1 and BOND*1, are maintained and may be used within the subroutine.

CHMTRN subroutines also have the feature that they pass back to the calling program a success or failure flag, using RETURN SUCCESS and RETURN FAIL. The success or failure of a subroutine may be tested in the calling program using

```
        IF SUCCESSFUL THEN ...
or      IF UNSUCCESSFUL THEN ....
```

Iterative Code

It is often necessary to iterate a section of code. The most common iterative structure in CHMTRN is the DO loop, which executes a block of qualifiers once for each atom or bond or ring in a set. For example, a SAVED*ATOM set may often contain more than one atom:

```
              FOR EACH ATOM IN SAVED*ATOM 1 DO AT*ZAP
                DELETE SPECIFIED*ATOM 1
AT*ZAP        ENDDO
```

The DO block is repeated as many times as there are atoms in SAVED*ATOM 1. This block includes only one command (DELETE SPECIFIED*ATOM 1) but could just as well encompass an entire series of qualifiers. The items in the set of atoms or bonds being iterated through are referred to (one at a time) as SPECIFIED*ATOM(BOND) 1 within the range of the DO block.

Two other iterative constructions, LOOP and BRANCH, provide even more sophisticated capabilities. LOOP causes looping on the members of a set, on values of a variable, or on paths:

```
        LOOP ON OLEFIN BONDS

or      LOOP ON VAR1 VALUES EQUALING FROM (3) UP*TO (10)

or      LOOP ON THE SHORTEST PATHS FROM ATOM*2  &
          TOWARDS SAVED*ATOM 2
```

Execution of the loop continues until a FINISHED, DISCONTINUE, REORIENT, or END*TRANSFORM is encountered, at which point control returns to the location of the LOOP statement, and looping commences on the next element of the controlling parameter.

The distinction between LOOP and DO is one of scope. A DO should be used for blocks of qualifiers that all refer to the same structure. A LOOP must be used when several different precursors may be generated by successive iterations.

Branching to successive locations to generate multiple goal and/or subgoal offspring structures is achieved with the BRANCH specifier. Like JUMP, BRANCH passes control successively to a list of addresses:

```
BRANCH TO LOC1 THEN*TO LOC2 THEN*TO LOC3
```

and, as with LOOP, execution of a BRANCH continues until a FINISHED, DISCONTINUE, REORIENT, or END*TRANSFORM is encountered.

Within a limb of a BRANCH, in a group chemistry table, END*TRANSFORM implies PERFORM THE MECHANISM AND DISPLAY THE PRECURSOR, so an explicit command PERFORM AND DISPLAY must not be issued just before an END*TRANSFORM.

DISCONTINUE causes the current limb of the BRANCH to be abandoned without any of the mechanism commands encountered being acted upon. Control passes to the next limb of the BRANCH.

Note that any information present before commencing a BRANCH, such as SAVED*ATOMs or FLAGs, is stored. This information is available within each limb of the BRANCH but any new information of this type generated within one limb is local only to that limb, as information stored when the BRANCH statement is encountered is restored at execution of a DISCONTINUE or END*TRANSFORM. Consider:

```
        SAVE AS 1 ALPHA TO ATOM*1 OFFPATH
        Qualifiers
        BRANCH TO LOC1 THEN*TO LOC2
LOC1    Qualifiers
        CLEAR SAVED*ATOM 1
        END*TRANSFORM
LOC2    Qualifiers
        DELETE SAVED*ATOM 1
        END*TRANSFORM
```

The status of the transform at the BRANCH statement (i.e. with SAVED*ATOM 1 containing ALPHA TO ATOM*1 OFFPATH) is restored when the BRANCH statement is encountered the second time and the jump to LOC2 is executed. Thus the CLEARing of SAVED*ATOM 1 in the first limb of the BRANCH does not cause a problem in the second limb.

Note, however, that LHASA does not save mechanism commands when a BRANCH statement is encountered. Thus, mechanism commands that may have been encountered before a BRANCH are not restored before the 2nd and successive limbs of the BRANCH, and mechanism commands within a particular limb are not available to other limbs of the BRANCH. To be safe, each limb of a BRANCH should contain a complete and self-contained mechanism.

While saved and specified atoms and bonds are actually sets, it is often necessary to have other sets available in CHMTRN. It is also sometimes convenient to use variables for

calculations of various sorts. The table writer may define variables and sets as required, simply by declaring their names at the top of the table:

```
ATKOUNT IS A VARIABLE
GOODATS IS A SET
BADATS IS AN ARRAY*OF*SETS DIMENSIONED AT 10.
```

Any name, preferably mnemonic, can be used, provided it is not a CHMTRN word or address label.

Arithmetic can be done on variables, which are all integers. Calculations use real double precision Fortran variables to accomodate large intermediate results and avoid round-off error, with the exception of integer division, which has its own operator (%). Results are rounded to the nearest integer before being returned to the CHMTRN table from the Fortran code.

Sets may be manipulated in a number of ways, logically AND'ed or OR'ed, augmented or diminished by insertion or removal of elements. The long-range search tables, e.g. HALAC, provide good examples of the uses of variables and sets.

Mechanism Commands at Multiple Locants

The FORTRAN code in LHASA assumes that the locants associated with CHMTRN mechanism commands (e.g. DELETE, JOIN, ATTACH, etc.) are single atoms or bonds. Thus, if the CHMTRN writer intends to have a mechanism command operate on a locant known to contain more than one atom or bond, a DO loop must be used, as in:

```
                FOR EACH ATOM IN SAVED*ATOM 1 DO ADD*OME
                    ATTACH AN ETHER TO SPECIFIED*ATOM 1
ADD*OME     ENDDO
```

If such a construction is not used, only the first atom in the SAVED*ATOM 1 set will get an OMe.

Transform Writing: Elements of Style

Transforms are now being added at a number of centers other than Harvard. The standard style and layout of qualifiers given below and in the sample transform for each table as shown in the section, “Specific Table and Transform Formats”, are commended to the reader in the pursuit of uniformity.

Starting Position

A qualifier, excluding any label that begins it, should start one tab position from the left margin. All labels (with the exception of BLKEND labels) should be written starting at the left margin:

```
            ADD 10 IF ATOM*1 IS BENZYLIC AND*THEN GO TO AT1*BZ
            KILL IF THERE IS MORE THAN ONE HYDROGEN ON ATOM*1
AT1*BZ      ADD 20 IF THERE IS NO HYDROGEN ON ATOM*1.
```

Continuation

CHMTRN lines can be of arbitrary length, but for readability should not exceed 72 characters. If a qualifier is too long, it may be continued on the next line by inserting two spaces and an "&"

at a logical breaking point and continuing on the next line after indenting three spaces from the previous indentation level. For example,

```
KILL IF THE APPENDAGES FROM CARBON2*2 TOWARDS  &
  SAVED*ATOM 1 AND FROM CARBON2*2 TOWARDS  &
  SAVED*ATOM 2 ARE NOT IDENTICAL.
```

It is considered poor form to break lines in awkward places, as in:

```
SUBTRACT 15 IF THERE IS A CARBONYL ON SAVED*ATOM  &
  5
```

Note that a continuation sign must not be used in GROUP*x statements. In this case only, continuation is automatic until the next Directory Set Operator is encountered, but a line must not end in an OR. Thus, the correct form is:

```
GROUP*1 MUST BE ALDEHYDE OR KETONE OR ESTER
  OR AMIDE*3 OR ACID
```

Positioning of Kill Qualifiers

Execution can be greatly sped up by killing transforms as early as possible. It is therefore advisable to order your qualifiers such that KILL qualifiers appear as near to the beginning of the transform as possible. If the features that KILL the transform are related to the keying elements for a goal transform, LHASA will often be able to use subgoal transforms to rectify these mismatches. Other mismatches may not be correctable, and the execution of subgoals in these cases would be unproductive. KILL qualifiers for these unsubgoalable mismatches should be placed at the top of a transform, before an END*PRESCREEN qualifier. When a goal transform fails before the END*PRESCREEN statement is encountered (if there is one), LHASA will remove it from consideration for future subgoal requests.

Address Labels

Any string of alphanumeric characters may be used as an address label in CHMTRN, provided that the label is not the same as a CHMTRN specifier. Previously, most labels were simply BLOCKx, for x=1,18. The current (and future, we hope) trend is to use names that are mnemonic for the blocks of code that they label.

DO Loops

The start and finish of a DO loop should be made obvious by correct indentation, as shown here:

```
          FOR EACH CARBON ALPHA TO ATOM*1 DO ALPH*C
            ADD 15 IF SPECIFIED*ATOM 1 IS ALLYLIC
ALPH*C    ENDDO
          SUBTRACT 5 IF THERE IS A DONATING GROUP ON ATOM*3
```

Nested DO loops should be written in an analogous manner. If a GO TO occurs within the DO loop, the label should be written at the left margin:

```
          Qualifiers
          IF ... DO LOC1
            Qualifiers
            IF ... DO LOC2
              Qualifiers
              IF ... THEN GO TO LOC3
              Qualifiers
LOC3          Qualifiers
LOC2        ENDDO
LOC1      ENDDO
          Qualifiers
```

Mechanism Commands

Mechanism commands should be enclosed between the two lines containing four dots (....) in a transform entry. These four-dot lines are important in that LHASA uses the position of the second of them to decide when to actually perform the atom and bond manipulations that it has accumulated in reading the mechanism commands. It should be kept in mind that a four-dot line is not an executable statement – it is not necessary for LHASA to actually read the second four-dot line before it performs the mechanism. In fact, the four-dot lines do not even appear in the binary versions of the transform tables that are interpreted by LHASA at run time. Thus, it is perfectly acceptable to have a GO TO in the table that jumps to a location somewhere after the second set of four dots. LHASA will realize that the four dots have been passed and perform the mechanism.

It is important that if an explicit PERFORM THE MECHANISM or PERFORM THE MECHANISM AND DISPLAY THE PRECURSOR command has been issued in a mechanism, there must always be further mechanism commands before the second four-dot line is passed. Thus an error would be given by the following code if there were an ester on ATOM*1:

```
Mechanism commands
PERFORM THE MECHANISM
IF THERE IS AN ESTER ON ATOM*1 THEN GO TO LOC1
Mechanism commands
....
```

The correct format would place the PERFORM command after the GO TO qualifier so that if there were an ester on ATOM*1, the PERFORM would never be encountered.

It should also be noted that it is perfectly acceptable to use any type of non-mechanism qualifier in the range of the mechanism. The first set of four dots is merely cosmetic. For example, it is often necessary to make certain mechanism commands conditional, as in

```
IF THE FIRST GROUP IS ESTER THEN  &
  BREAK SAVED*BOND 1 AND*THEN GO TO LOC4
```

Combination of Qualifiers

The two qualifiers

```
      KILL IF THERE IS AN ESTER ON ATOM*2
and   KILL IF THERE IS AN ESTER ON ATOM*3
```

should be written as one qualifier for faster execution:

```
KILL IF THERE IS AN ESTER ON ATOM*2 OR: ON ATOM*3.
```

Note that the combining word in this case is OR: (logical OR) and not AND: (logical AND). Always be sure you have the right one. In this case the intention is that the transform be killed if either ATOM*2 or ATOM*3 is the carbon origin of an ester. If AND: had been used, both ATOM*2 **and** ATOM*3 would have had to be ester origins for the qualifier to be true.

OR: and AND: can be used to join together qualifiers in which there is a constant target or locant. This target or locant specifier must appear in the first portion of the qualifier, and, if the CHMTRN will allow it, the operation specifier (e.g. KILL, ADD, or SUBTRACT) should be placed first of all.

It is possible to combine qualifiers in which both the target and the locant vary by using the OR:IF and AND:IF logical connectors. Thus the two qualifiers

```
      KILL IF THERE IS AN ESTER ON ATOM*3
and   KILL IF THERE IS A NITRILE ON ATOM*4
```

could be combined into:

```
KILL IF THERE IS AN ESTER ON ATOM*3  &
  OR:IF THERE IS A NITRILE ON ATOM*4.
```

Combinations that contain OR:, AND:, and AND:IF, or those that contain OR:, AND:, and OR:IF are legal. It is not possible, however, to combine both OR:IF and AND:IF in the same qualifier. For example, the following would be a legal qualifier:

```
KILL IF THERE IS A NITRILE ON ATOM*1  &
  OR: ATOM*2 AND: ATOM*6 AND:IF  &
  THERE IS A HALIDE ON ALPHA TO ATOM*3 OFFPATH
```

This qualifier requires that there be simultaneously a halide on the atom alpha to ATOM*3, a nitrile on ATOM*6, and a nitrile on either ATOM*1 or ATOM*2, in order to kill the transform.

When FOR EACH qualifiers are combined, the AND:FOR specifier (a synonym of OR:IF) is used to improve readability:

```
ADD 15 FOR EACH HALOGEN*ATOM ALPHA TO ATOM*2  &
  AND:FOR EACH ATOM IN HETSET.
```

AND does not combine qualifiers, but rather separates two CHMTRN words that would otherwise be wrongly packed into the same field in the same word, as in:

```
JOIN ATOM*1 AND ATOM*6.
```

OR functions as a word separator in GROUP statements, e.g.

```
GROUP*1 MUST BE ACID OR ESTER
```

or as a separator for numerical arguments used with LARGER:

```
KILL IF BOND*1 IS IN A RING OF SIZE 8 OR LARGER.
```

Buzz Words

It is possible to write very terse, yet correct, CHMTRN. For example, LHASA will interpret the following qualifier correctly:

```
KILL NOT ESTER ALPHA ATOM*3 OFFPATH.
```

There is a certain virtue in the liberal use of buzz words, however. Buzz words were included in the language to make CHMTRN qualifiers sound as much like normal English as possible. It is the responsibility of each transform writer to ensure that reading CHMTRN be as natural as possible for the uninitiated chemist. Accordingly, the above qualifier should be written:

```
KILL IF THERE IS NOT AN ESTER ON  &
  ALPHA TO ATOM*3 OFFPATH.
```

Comments

It is also the case that an amply commented transform is more useful to future readers than an esoteric entry with various indecipherable qualifiers in it. If a block of CHMTRN code is included for a specific purpose, write a comment explaining the reason for its inclusion. If it's not obvious why a qualifier is there, a judicious comment can never hurt. It is legal to put a comment on the same line as its qualifier, in fact, as long as three dots precede it.

Packing Efficiency

As mentioned above, it is inefficient to have two or more consecutive qualifiers with the same target or the same locant. There are frequently other cases where packing efficiency can be optimized. A good guideline to follow is that a normal qualifier contains, in this order an operation specifier, a target specifier, and a locant specifier, as in:

```
KILL IF THERE IS AN ESTER ON ATOM*1
  |                          |          |
Operation                  Target     Locant
```

The times when it is necessary to deviate from this guideline are usually when the target is qualified by a number in the QLINCR field and the operation involves an ADD or SUBTRACT. Thus:

ADD 10 IF THERE ARE TWO HYDROGENS ON ATOM*2

would try to pack both 10 and TWO into the QLINCR field. The correct qualifier is:

IF THERE ARE TWO HYDROGENS ON ATOM*2 THEN ADD 10.

Clearly, a qualifier like:

IF THERE IS AN ESTER ON ATOM*1 THEN KILL

can be packed into one CHMTRN word if rewritten.

Here's My Transform – Where Do I Put It?

Having collected information about a reaction, you then have to decide which knowledge-base table to put the new transform into. As mentioned earlier, the LHASA knowledge base is composed of two main parts, the transforms and the long-range searches. The decision between these two parts is fairly easy – if the library work for the reaction took a day to a week, write a transform; if it took a month to a year, write a long-range search.

Assuming you haven't just spent a year in the library, let's talk about the transform knowledge base. As mentioned above, there are several different transform-oriented knowledge-base tables. These tables may also be divided into two groups, the goal transforms and the subgoal transforms (see Figure 6.1). Brief descriptions of the tables are given in Figure 6.2. Subgoal transforms are easily recognized: they can be keyed by a single functional group (or by no functional group) and they do not perform a major simplification of the target structure. The tables in this category are: RFGTB, which takes reactive functional groups back retrosynthetically to core functionality ("reactive" functional groups in LHASA are, for example: ACID*HALIDE, DIAZO, and IMINE); FGITBx, whose transforms interchange one functional group for another; FGATBx, which adds functionality on unfunctionalized centers; and FGRTB, which removes functionality from functionalized centers. Examples of these four non-simplifying transform types are shown in Figure 6.3.

Transform Tables		**Long-range Search Tables**
Goals	**Subgoals**	
		DLSTB
SGTABx	RFGTB	ROBIN
PAIRTx	FGITBx	HALAC
OLFTB	FGATBx	BIRCH
PCHEMx	FGRTB	DZOTB
		QDATB

Figure 6.1 Schematic Representation of the Knowledge Base

Transform Tables

SGTABx - transforms keyed by single functional groups
PAIRTx - transforms keyed by pairs of functional groups
OLFTB - stereospecific olefin syntheses
PCHEMx - transforms keyed by complex substructures
RFGTB - transforms converting reactive functional groups to core functionality
FGITBx - functional group interchanges
FGATBx - functional group additions
FGRTB - functional group removals

Long-range Search Tables

DLSTB - Diels-Alder
ROBIN - Robinson Annulation
HALAC - Halolactonization, Hydroxylactonization
BIRCH - Birch Reduction
DZOTB - Alpha-diazoketocarbene Cyclopropanation
QDATB - Quinone Diels-Alder

Figure 6.2 Brief Descriptions of LHASA Knowledge-base Tables

<.figure 50>

Figure 6.3 Examples of LHASA Subgoal Transforms

Goal transforms in LHASA are ones that perform major simplifications of the target structure, either by reducing the level of functionalization, disconnecting a bond, removing an appendage, simplifying stereochemistry, etc. There are four subcategories of goal transforms: olefin, one-group, two-group, and 1-D-pattern. Stereospecific olefin syntheses go into OLFTB. Disconnective transforms keyed by a single functional group, e.g.

<.figure 10>

go into the one-group tables SGTABx.

Simplifying transforms keyed by a pair of functional groups, e.g.

<.figure 10>

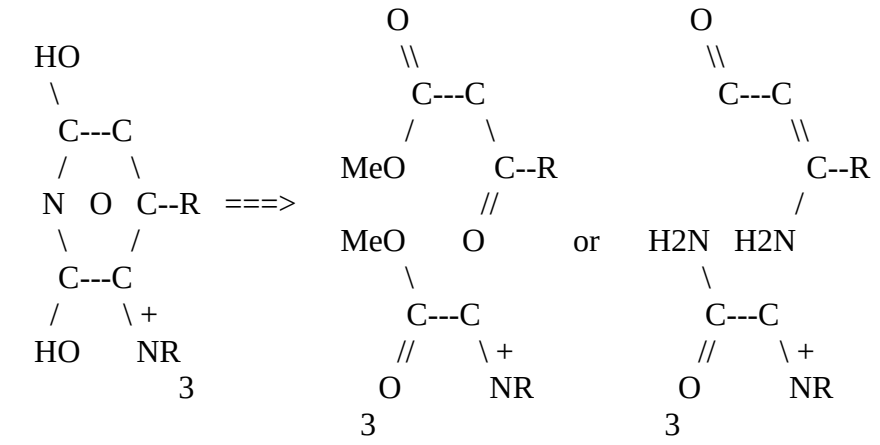
go into the two-group tables PAIRTx.

All other goal transforms, which in general will be those keyed by more complex substructures (e.g. aromatic rings, heterocyclic rings, etc.) go into one of the pattern chemistry tables (PCHEM1, PCHEM2, etc.). A sample pattern-keyed transform might be the Guareschi-Thorpe pyridone synthesis:

TRANSFORM 1474
NAME Guareschi-Thorpe Pyridone Condensation

REFERENCES

Ref: Klingsberg, Vol. 14, part 3
Author: Tony Baxter ICI 11/4/84



END*REFERENCES
RATING 55
1D*PATTERN
N[EPS=1;HETS=0]%C(&O)%C(N[CHARGE=CATION;HETS=0;HS=0])%
C[HETS=0]%C[HS=1]%C(&O)%@1
END*PATTERNS
COPYRIGHT*LUK
DISCONNECTIVE
...
IF ALPHA TO ATOM*3 OFFPATH IS CARBON OR: HETERO THEN KILL
IF ALPHA TO ATOM*8 OFFPATH IS CARBON OR: HETERO THEN KILL
....
BRANCH TO CRBONYL THEN*TO ENAMIN
CRBONYL CONDITIONS NaOR AND NH3
BREAK BOND*1
ATTACH AN ETHER TO ATOM*2
SINGLE BOND*6
ATTACH A CARBONYL TO ATOM*6
GO TO CMNMECH
ENAMIN CONDITIONS NaOR
SINGLE BOND*1
ATTACH AN AMINE*2 TO ATOM*6
DOUBLE BOND*6
BOND*7 IS DEFINED*SYN TO THE NEWEST*BOND ON ATOM*6
CMNMECH SINGLE BOND*3
BREAK BOND*5
SINGLE BOND*7
BREAK BOND*9
DOUBLE BOND*2

DOUBLE BOND*8
ATTACH AN ETHER TO ATOM*8
....

In general, searching the group-oriented tables for transforms is faster than searching the pattern tables, so, if you can, make it a group-keyed transform. Pattern transforms have the advantage of allowing longer paths (ATOM*1 to ATOM*n, BOND*1 to BOND*n), however, which can be very convenient in the case of a cyclic or otherwise complex keying substructure.

How is a Transform Rated?

Ratings for transforms have caused a considerable amount of discussion among LHASA practitioners over the years. Until recently, little attempt has been made to correlate ratings from one transform to the next. Thus, ratings have been useful principally for comparison of the applicability of a particular transform to two or more different targets; their usefulness in comparing the virtues of two different transforms applied to the same target has been limited.

There are really two types of transform ratings in LHASA, the "initial rating" and the "final rating". The initial rating reflects the merit of a transform in relation to other transforms in the knowledge base without regard for the target structure. The final rating is calculated by summing the initial rating and any increments or decrements encountered during transform evaluation.

Final ratings are used in LHASA to decide whether or not to show a transform to the chemist. If too many SUBTRACT qualifiers are executed, the final rating for the transform can fall below the rating cutoff, and the transform will be killed. Final ratings are calculated on a scale of 0 to 100. A transform with a calculated increment of 0 or less will not be shown to the chemist. Final ratings of more that 100 are reset to 100. Increments for particular structural features are unfortunately not standardized from one transform to the next. This is an area for future improvement to the knowledge base.

Initial ratings, used principally for assessing the utility of multi-step subgoal sequences, are now standardized throughout the entire transform knowledge base according to a newly-developed scheme, described below. Initial ratings are calculated by the CHMTRN compiler from qualitative ratings for a transform in each of eight categories. These qualitative ratings, expressed as CHMTRN keywords, are translated to numbers by the CHMTRN compiler, summed (each category also has a weight associated with it), and scaled into the range 0 - 100 to give the initial transform rating.

Descriptions of the eight categories, their default weights, and scales for the qualitative rating keywords are given below. Note that the numerical equivalents of BAD, POOR, FAIR, GOOD, VERY*GOOD, and EXCELLENT are 0, 1, 2, 4, 6, and 8, respectively.

1) TYPICAL*YIELD (Default weight 10)

BAD	0-40%
POOR	41-60%
FAIR	61-70%
GOOD	71-80%
VERY*GOOD	81-90%
EXCELLENT	>90%

2) RELIABILITY (Default weight 8)

The Reliability is a measure of the predictability of the reaction. Stereoselectivity, if it is a factor, can also be included in this category. If there are many factors which must be taken into account when the transform is evaluated, this index should be low. A reaction which gives uniformly poor yields, but can be relied upon to always give something should be given a high reliability index.

BAD	Yields and/or stereoselectivity vary widely for no well-known reason
POOR	Predicted yields and/or stereoselectivity frequently more than 25% off
FAIR	Wide variety of yields and/or stereo-selectivities, but factors leading to variation are well known
GOOD	Most known examples fall within 20% yield and stereoselectivity range
EXCELLENT	Most known examples fall within 10% yield and stereoselectivity range

3) REPUTATION (Default weight 5)

Well-known reactions with many examples in the literature should be given a high reputation index.

BAD	One article with five or fewer examples
POOR	One article with more than five examples
FAIR	Two to eight articles or no general review
GOOD	More than eight articles, at least one good review
EXCELLENT	Several review articles

4) HOMOSELECTIVITY (Default weight 4)

Homoselectivity is the property of selectivity among several different functional groups of the same type. For example, if a reaction can be performed on a primary alcohol in the presence of a secondary alcohol, it has high homoselectivity. Note that reactions of relatively rare functional groups (e.g. enol ether) should not be penalized severely in this category, since there are not often multiple occurrences of these groups in the reactant.

BAD	No precedent for preference
POOR	Some selectivity between functional groups in differing environments
FAIR	Better than 5:1 selectivity in widely differing environments, e.g. primary vs. tertiary alcohols
GOOD	Better than 5:1 selectivity in moderately differing environments, e.g. primary vs. secondary alcohols
EXCELLENT	Better than 5:1 selectivity between fairly similar cases, e.g. primary alcohols with and without alpha branching

5) HETEROSELECTIVITY (Default weight 4)

Heteroselectivity is the property of selectivity among several possible modes of reactivity. If a reaction has few interfering functional groups, then it has high heteroselectivity.

BAD	Most other FGs reactive to necessary reaction conditions
POOR	Many common FG types incompatible

FAIR	Less than 20% of FG types incompatible
GOOD	Less than 10% of FG types incompatible
EXCELLENT	Less than 5% of FG types incompatible

6) ORIENTATIONAL*SELECTIVITY (Default weight 4)

When different orientations at a single reaction site are possible, this index reflects the selectivity between them. For example, an addition to an olefin is given a low orientational selectivity index if electronic and steric factors have no effect on the orientation of the addition. An addition with highly predictable orientational preferences is given a high index. Symmetrical additions and reactions which can only occur in a single orientation should be rated as NOT*APPLICABLE (equivalent to EXCELLENT).

BAD	Typically 1:1 mixtures on unsymmetrical substrates
POOR	2:1 mixtures or better when sufficiently differentiated, e.g. gem-disubstituted olefins
FAIR	5:1 or better obtainable in some cases, e.g. gem-disubstituted olefins
GOOD	5:1 or better obtainable in similar cases, e.g. monosubstituted or trisubstituted olefins
EXCELLENT	5:1 or better obtainable in most unsymmetrical substrates.
NOT*APPLICABLE	Only one orientation possible.

7) CONDITION*FLEXIBILITY (Default weight 3)

The flexibility index of a reaction describes the degree of choice the chemist has in choosing conditions for the reaction. If several reagents are available with varying reactivities in different cases, the reaction has high flexibility.

BAD	Only one set of conditions available
POOR	Two or three similar sets of conditions
FAIR	Several quite similar sets of conditions
GOOD	Three or four quite different sets of conditions
EXCELLENT	Five or more quite different sets of conditions

8) THERMODYNAMICS (Default weight 2)

This parameter reflects the intrinsic yield for the reaction, irrespective of external factors such as workup, selectivity, etc. A high thermodynamic index indicates that the reaction may proceed in very high yield in certain very favorable cases.

BAD	Unreacted starting material must be recycled to get better than 30% yields
POOR	Unreacted starting material must be recycled to get better than 60% yields
FAIR	10 - 30% starting material unreacted
GOOD	1 - 10% starting material unreacted
EXCELLENT	Less than 1% starting material unreacted

As mentioned above, the initial rating keywords are included in the header information for each transform. Thus, the header for the Grob Fragmentation transform would look like:

```
        TRANSFORM 3
        NAME Grob Fragmentation

... March 775; JACS 76, 2285; 2291; 2294 (1954);
... JACS 78, 4057 (1956);
... ACIE 6, 1 (1967); JACS 101, 3567 (1979)
...
        TYPICAL*YIELD                   GOOD
        RELIABILITY                     EXCELLENT
        REPUTATION                      GOOD
        HOMOSELECTIVITY                 GOOD
        HETEROSELECTIVITY               GOOD
        ORIENTATIONAL*SELECTIVITY       EXCELLENT
        CONDITION*FLEXIBILITY           GOOD
        THERMODYNAMICS                  EXCELLENT
...
...             O       OH,NH2   OH
...             "        |        |
... C==C + C   =>  C-----C----C
...       |                |
...       H                H
...
```

Using the category weights and the numerical equivalents of the qualitative keywords as described above, the Grob Fragmentation would get an initial rating of 68. LHASA would round this number off to the nearest multiple of 5, i.e. 70. It should also be noted that the rating scheme described above is not yet used in every LHASA transform. While conversion to the new system is taking place, LHASA will be able to handle either the new scheme or the old scheme, in which the initial rating is simply declared in a RATING statement.

Specific Table and Transform Formats

Overall Transform Table Organization

Before the actual transforms in a transform table, there is usually a certain amount of information pertaining to the table as a whole. If a non-standard abbreviation for a literature reference has been used, that abbreviation should be explained:

```
        ...B+P = Buehler, Calvin A. and Pearson, Donald E.,
        ..."Survey of Organic Syntheses",
        ...Wiley-Interscience, New York, NY, 1970.
```

Any GLOBAL code blocks used in the table should then be listed, with brief explanations and locations:

```
...GLOBAL*12 checks for identicality and symmetry
...    in aldol precursors, TRANSFORM 102
```

The range of transform numbers appropriate for the table is given next. The availability of actual transform numbers within this range must be checked in LHXL:TRNSFM.DAT when selecting a number for a new transform. Available numbers are listed in TRNSFM.DAT as "Reserved for future XXXXX transform".

If any CHMTRN variables or sets are used in the table, they should be defined in the first non-comment lines read by the CHMTRN compiler. A variable or set name must not already be in use as a label name or CHMTRN word. Subject only to this restriction, variable and set names should be as mnemonic as possible. Note that it is necessary to declare variables and sets used not only in the transforms themselves, but also in any subroutines called by these transforms, at the top of each table.

```
PATHEND
FIN
```

These are the common features of transform tables. In addition, each transform table has a number of specific features that will now be described. It is important to remember these features when adding to an existing table or when creating a small table for testing a new transform.

Individual Transform Tables

PAIRTx

The choice of table for transforms keyed by pairs of functional groups is governed by the length of the path in bonds between the two groups. Thus, in a transform where the number of bonds between the two groups is not significant, such as Ozonolysis, the transform is PATH INDEPENDENT and belongs in PAIRTI. If the two groups share a common origin:

<.figure 10>

as in the Halogenation of Nitro, Sulfoxide, or Sulfone transform, thetransform is "Path 0" and would be found in PAIRT0. However, when the origins are distinct, as in:

O = C - C - C - C - C = C

for the Michael Reaction, the bonds on the path joining the carbon origins must be counted ("Path 4" in this example), and the transform would be in PAIRT4.

PAIRTx tables use transform numbers 1 - 399. A typical PAIRTx transform header looks like:

```
      TRANSFORM 1
      NAME Oxidative Olefin Cleavage
...AOS 11; March 871; House 358; B+P 215
...
      TYPICAL*YIELD                VERY*GOOD
      RELIABILITY                  GOOD
```

```
        REPUTATION                      EXCELLENT
        HOMOSELECTIVITY                 GOOD
        HETEROSELECTIVITY               EXCELLENT
        ORIENTATIONAL*SELECTIVITY       EXCELLENT
        CONDITION*FLEXIBILITY           GOOD
        THERMODYNAMICS                  EXCELLENT
...
... O         O
...  "         "
... C    +    C   =>    C=C
..  |         |         |  |
... O,NR2  O,NR2   H  H
...
        SAME*RATING (Note 1)
...
... O         O
...  "         "
... C +  C---C => C=C-C
...  |         |      |  |
... O,NR2   C     H  C
...
        SAME*RATING
...
... O=C + C=O => C=C
...
        ...PATH INDEPENDENT         (Note 2)
        GROUP*1 MUST BE ALDEHYDE OR KETONE OR ESTER
          OR AMIDE*3 OR ACID        (Note 3)
        GROUP*2 MUST BE ALDEHYDE OR KETONE OR ESTER
          OR AMIDE*3 OR ACID
        SUBGOAL*1 MUST BE ALDEHYDE OR KETONE OR ESTER
          OR AMIDE*3 OR ACID        (Note 4)
        SUBGOAL*2 MUST BE ALDEHYDE OR KETONE OR ESTER
          OR AMIDE*3 OR ACID
Special Sets      (Note 5)
        BROKEN*BONDS BOND1*1 BOND2*1
        ...
        Target Qualifiers
        ....
          Mechanism Commands
        ....
        Precursor Qualifiers
```

PAIRTx Notes

1 This particular transform has multiple 2-D patterns, each accompanied by its own rating specification. In the event that the ratings are the same in every category as those for the previous pattern, the short-cut keyword SAME*RATING can be used.

2 There are seven possible path lengths in the PAIRTx tables, as described above.

3 Every two-group transform must have one or more GROUP*1 and one or more GROUP*2 keying group types specified.

4 Two-group transforms deemed powerful enough to request subgoals must contain SUBGOAL*1 and SUBGOAL*2 sets analogous to the GROUP*i sets. The group types specified in each SUBGOAL*i set must be a subset of those in the corresponding GROUP*i set.

5 The Special Sets available in the PAIRTx tables are:

DISCONNECTIVE EXTENDED*SUBGOAL INTERMOLECULAR
NON*OPPORTUNISTIC RECONNECTIVE RING*REQUIRED
STEREOSELECTIVE STUDENT SYMMETRICAL UNMASKING

OLFTB

The olefin table, OLFTB, is complicated in that it contains both a binary search section (used for prescreening) and a transform section (details are available in E.J. Corey and A.K. Long, J. Org. Chem. 43 2208 (1978)). The head of the table also contains CHMTRN code for filling the perception set for the target structure:

Perception set code
Subroutines
Perception set code
Transforms
Subroutines
OLEFIN*SETS
FIN

The PATHEND used in normal group-oriented tables is replaced here by OLEFIN*SETS. This specifier tells the CHMTRN program to construct tables of transform and screening set locations. OLFTB uses transform numbers 400 - 449. A typical OLFTB transform header looks like:

```
    TRANSFORM 410
    NAME Claisen Rearrangement
...TL 3243 (1969); JACS V.92 741, 4461, 4463 (1970)
...JACS V.95 553 (1973)
...
    TYPICAL*YIELD                GOOD
    RELIABILITY                  GOOD
    REPUTATION                   EXCELLENT
    HOMOSELECTIVITY              GOOD
    HETEROSELECTIVITY            EXCELLENT
    ORIENTATIONAL*SELECTIVITY    EXCELLENT
    CONDITION*FLEXIBILITY        POOR
    THERMODYNAMICS               EXCELLENT
...
```

```
...              O              OH            OR
...              "              |             |
... C=C-C-C-C     =>     C-C=C + C=C
...              |                            |
...              C,H,OR,NR2                   C,H,OR,NR2
...
```

...Kill specifiers for prescreen (Note 1)
RINGBOND TRANS2, FUNCTIONAL*GROUP TRANS1
NO*FG*WITHIN*ALPHA TRANS3
OLEFIN*TYPE TETRASUBSTITUTED Z*DISUBSTITUTED
SPACING*1*5 WELL*DEFINED*T*FIRST WELL*DEFINED*L*LAST

...

Target Qualifiers
....
Mechanism Commands
....
Precursor Qualifiers

OLFTB Notes

1 KILL qualifiers within the body of OLFTB transforms are largely replaced by "Kill Specifiers" as shown. The way in which these prescreening specifiers is used is described in the leading comments in OLFTB.

Keying groups, path lengths, and special sets are not used in OLFTB transforms.

RFGTB

All transforms in RFGTB have the same path length. Transform numbers are in the range 450 - 499. A typical RFGTB transform header looks like:

TRANSFORM 450
NAME Diazo via Hydrazone or Oxime
...S&K 1,391; F+F 1,655
...
TYPICAL*YIELD FAIR
RELIABILITY GOOD
REPUTATION GOOD
HOMOSELECTIVITY FAIR
HETEROSELECTIVITY GOOD
ORIENTATIONAL*SELECTIVITY EXCELLENT
CONDITION*FLEXIBILITY GOOD
THERMODYNAMICS EXCELLENT
...
... C=N2 => C=O
...
GROUP*1 IS DIAZO
STUDENT
...
Target Qualifiers
....

```
   Mechanism Commands
....
Precursor Qualifiers
```

RFGTB Notes:

The only Special Set used by this table is STUDENT, which allows the transforms to be used by the student version of LHASA.

SGTABx

There are three SGTABx tables, SGTAB1, SGTAB2, and SGTAB3. For transforms in the single-group table SGTAB1, a minimum path of one bond is always grown. These transforms are subdivided into two categories: "Path 0" and "Path 1". The distinction between these subdivisions is that Path 0 transforms break a bond **in** the keying functional group, while Path 1 transforms break a bond **adjacent** to the functional group. These latter transforms must contain the special set label PATH*ONE*SET in their table entries.

Transforms in the other two SGTABx tables (SGTAB2 and SGTAB3) enjoy a more normal correspondence between table name and path length.

To test a new one-group transform prior to its inclusion in the main table, simply put it into an appropriately named dummy table (e.g. SGTAB2.SRC) and CHMTRN as usual.

SGTABx tables use transform numbers 500 - 699. A typical SGTABx transform header looks like:

```
TRANSFORM 500
NAME Carbonium Ion Formation
... March 140; House 809
...
TYPICAL*YIELD                 VERY*GOOD
RELIABILITY                   FAIR
REPUTATION                    EXCELLENT
HOMOSELECTIVITY               GOOD
HETEROSELECTIVITY             FAIR
ORIENTATIONAL*SELECTIVITY     EXCELLENT
CONDITION*FLEXIBILITY         FAIR
THERMODYNAMICS                FAIR
...
...  PLUS
...  |
... C   => C-OH
..
...PATH 0 BONDS        (Note 1)
GROUP*1 MUST BE CARBONIUM
Special Sets        (Note 2)
CREATES*STEREO CARBON1*1
...
Target Qualifiers
....
  Mechanism Commands
....
Precursor Qualifiers
```

SGTABx Notes

1 There are four possible path lengths in the SGTABx tables:
Path 0, where the broken bond (if any) is **in** the keying group,
Path 1, where the broken bond is **adjacent** to the keying group,
Path 2 and Path 3, where the broken bonds are, respectively, two and three bonds away from the origin of the keying functional group.

2 The Special Sets available in SGTABx tables are:
EXTENDED*SUBGOAL INTERMOLECULAR ORIGIN*INDEPENDENT
NON*OPPORTUNISTIC PATH*INDEPENDENT PATH*ONE*SET
REARRANGEMENT RING*REQUIRED STEREOSELECTIVE
STUDENT UNMASKING

FGATBx

As with the FGI tables, there are two functional group addition tables, FGATB0 and FGATB1. Path change 0 FGA transforms add a functional group using an existing carbon atom as the new origin:

<.figure 8>

Path change +1 additions add a functional group **and** its carbon origin, as in:

<.figure 8>

FGATBx tables use transform numbers 700 - 749. A typical FGATBx transform header looks like:

```
        TRANSFORM 700
        NAME Decarboxylation
...March 477; House 511; B+P 677; Tet. Lett. 2243 (1977)
...
        TYPICAL*YIELD                   EXCELLENT
        RELIABILITY                     EXCELLENT
        REPUTATION                      EXCELLENT
        HOMOSELECTIVITY                 BAD
        HETEROSELECTIVITY               EXCELLENT
        ORIENTATIONAL*SELECTIVITY       EXCELLENT
        CONDITION*FLEXIBILITY           GOOD
        THERMODYNAMICS                  EXCELLENT
...
... H      COOH
...  |      |
... C => C
...  |      |
... W    W
...
        TYPICAL*YIELD                   POOR    (Note 1)
        RELIABILITY                     FAIR
```

REPUTATION FAIR
HOMOSELECTIVITY POOR
HETEROSELECTIVITY FAIR
ORIENTATIONAL*SELECTIVITY EXCELLENT
CONDITION*FLEXIBILITY POOR
THERMODYNAMICS EXCELLENT

...
... H COOH
... | |
... C => C
...

...Path Change +1 (Note 2)
GROUP*1 IS ACID (Note 3)
Special Sets (Note 4)
...
Target Qualifiers
....
Mechanism Commands
....
Precursor Qualifiers

FGATBx Notes

1 This transform, like the example in the subsection, “PAIRTx”, has multiple 2-D patterns, each with its own rating block.

2 As described above, the path change determines the functional group addition table in which the transform belongs.

3 GROUP*1 in the FGATBx tables is the group type (or types) resulting from the operation of the transform. In other words, GROUP*1 specifies the requestable group types for a given transform.

4 The Special Sets available in FGATBx are:
LAST*RESORT STUDENT

FGITBx

There are two functional group interchange (FGI) path lengths (tables FGITB0 and FGITB1). These path lengths represent the change in position of the functional group origin between target and precursor. Thus, the synthetic reduction of a ketone to an alcohol:

<.figure 10>

is "Path Change 0" as the functional group origin does not move. However, a 1,2-Carbonyl Transposition:

<.figure 10>

is "Path Change 1", since the origin has moved by one carbon.

There is a further subdivision of each FGITBx table for cosmetic rather than functional reasons. The transforms are organized according to the oxidation level of the OBJECT GROUP and then of the SUBJECT GROUP. These subdivisions are clearly explained in comment statements in the tables. Please adhere to this subcategorization when adding new transforms.

FGITBx tables use transform numbers 750 - 949. A typical FGITBx transform header looks like:

```
        TRANSFORM 750
        NAME Ester Hydrolysis
...March 309; House 511,513; B+P 748; F+F 1, 617
...
        TYPICAL*YIELD                   EXCELLENT
        RELIABILITY                      EXCELLENT
        REPUTATION                       EXCELLENT
        HOMOSELECTIVITY                 FAIR
        HETEROSELECTIVITY               GOOD
        ORIENTATIONAL*SELECTIVITY       EXCELLENT
        CONDITION*FLEXIBILITY           GOOD
        THERMODYNAMICS                  GOOD
...
... O           O
... "            "
... C-OH => C-OR
...
        ...Path Change 0   (Note 1)
        SUBJECT GROUP IS ACID      (Note 2)
        OBJECT GROUP IS ESTER      (Note 3)
        Special Sets         (Note 4)
        BROKEN*BONDS BOND1*1
...
        Target Qualifiers
        ....
          Mechanism Commands
        ....
        Precursor Qualifiers
```

FGITBx Notes

1 As described above, the path change determines which functional group interchange table the transform belongs in.

2 The SUBJECT GROUP in the FGITBx tables is the keying group for the transform. A transform may have multiple subject groups, as in:

```
SUBJECT GROUP IS ACID OR ESTER
```

3 The OBJECT GROUP in the FGITBx tables is the functional group type requested by the SUBGOAL*i (i = 1,2) specifier in a group-oriented goal transform table or by an EXCHANGE command. Like subject groups, object groups may be multiple.

To avoid having to write mechanism commands for each possible object group type, the REQUESTED specifier may be used. For example, if a long-range search table contains the functional group interchange request:

EXCHANGE THE GROUP FOR AN IODIDE ON ATOM*3

the FGITB0 transform SN2 Displacement might look like:

```
SUBJECT GROUP IS ???
OBJECT GROUP IS FLUORIDE OR CHLORIDE OR
  BROMIDE OR IODIDE
.
.

.
IF THE REQUESTED GROUP IS CHLORIDE  &
  THEN CONDITIONS ???
.
.
.
....
  REMOVE THE FIRST GROUP
  ATTACH THE REQUESTED GROUP TO ATOM*1
```

4 The Special Sets available in the FGITBx tables are:

DISCONNECTIVE LOW*UTILITY MEDIUM*UTILITY RECONNECTIVE STEREOSELECTIVE STUDENT

FGRTB

All transforms in FGRTB have the same path length. Transform numbers are in the range 950 - 969. A typical FGRTB transform header looks like:

```
    TRANSFORM 950
    NAME Benzylic Oxidation to Acid
... D.G.Lee, Oxidation of Organic Compounds by
...      Permanganate Ion and Hexavalent Chromium,
...      Open Court Publishing (1980)
...
    TYPICAL*YIELD                VERY*GOOD
    RELIABILITY                  GOOD
    REPUTATION                   EXCELLENT
    HOMOSELECTIVITY              POOR
    HETEROSELECTIVITY            FAIR
    ORIENTATIONAL*SELECTIVITY    EXCELLENT
    CONDITION*FLEXIBILITY        POOR
    THERMODYNAMICS               EXCELLENT
...
... Ar-COOH   =>   Ar-CH3
...
```

GROUP*1 MUST BE ACID (Note 1)
...
Target Qualifiers
....
Mechanism Commands
....
Precursor Qualifiers

FGRTB Note

1 GROUP*1 is the group to be removed.

There are no special sets for this table.

TAUTOMER

All transforms in TAUTOMER have the same path length. Transform numbers are in the range 970 - 999.

```
TRANSFORM 970
NAME Keto-enol Tautomerization
...STARTP (Note 1)
...C[HS>0;FGNOT=OLEFIN,ALDEHYDE,IMINE,OXIME,HYDRAZONE,
...EPOXIDE,AZIRIDINE,EPISULFIDE,DIAZO,ALLENE]-
...X[HETS=1,2]=O,S,N[HS=0] (Note 2)
...ENDP (Note 3)
RATING 50 (Note 4)
...
Target Qualifiers
....
Mechanism Commands (Note 5)
....
Precursor Qualifiers
```

TAUTOMER Notes

1 STARTP is a marker for PATRAN, the offline program that parses the patterns and creates the data structures that LHASA reads during execution. STARTP marks the beginning of the pattern. The lines from STARTP to ENDP, inclusive, must not be indented.

2 The pattern follows STARTP. The 1-D pattern in a TAUTOMER transform corresponds to the uncommon tautomeric form(s), the one(s) that do not appear as keying patterns in the transform knowledge base. Detailed instructions for writing 1-D patterns are included in the section, “Writing 1-D Patterns for LHASA”.

3 ENDP is the marker to PATRAN for the end of the pattern.

4 Ratings and rating modifications are not typically used in evaluating TAUTOMER transforms, so a default initial rating of 50 is used.

5 Mechanism commands in TAUTOMER transforms are those necessary to convert the uncommon tautomeric forms to the forms that appear in the knowledge base as keying patterns for pattern chemistry transforms.

There are no special sets for this table.

PCHEMx

Pattern chemistry knowledge-base tables (PCHEMx) are not subdivided into path lengths. The path information is contained in the transforms themselves.

Pattern chemistry transforms are numbered from 1001-2999. A typical PCHEMx transform header looks like:

```
        TRANSFORM 1027
        NAME N-Allylcarboxamide Pyrrole Synthesis
...N. Engel et.al. ACIE 676 1978[17]
...
...                          R'          R"
...  R'          R"          \          /
...   \          /             =====CH            CH3
...    \_____/                 |                   |
...    ||     ||    ====>      |       Cl    +   K-O-C-CH3
...    ||     ||               CH2      |          |
...     \     /\                \      //\         CH3
...      \   /   \                \  //    \
...       N     R                  N      R     (Note 1)
...
...STARTP       (Note 2)
...N[HS=1]%C(-C[HS=0;FGNOT=LEAVING,XWITHDRAWING])%
...C[HETS=0]%C[HETS=0]%C[HS=1;HETS=1]%@1        (Note 3)
...ENDP (Note 4)
        TYPICAL*YIELD                   GOOD
        RELIABILITY                     GOOD
        REPUTATION                      FAIR
        HOMOSELECTIVITY                 POOR
        HETEROSELECTIVITY               FAIR
        ORIENTATIONAL*SELECTIVITY       EXCELLENT
        CONDITION*FLEXIBILITY           POOR
        THERMODYNAMICS                  GOOD
        Special Sets        (Note 5)
        BROKEN*BONDS BOND*3
        ...
        Target Qualifiers
        ....
          Mechanism Commands
        ....
        Precursor Qualifiers
```

PCHEM notes

1 A descriptive picture is often included at the top of a pattern chemistry transform since the keying substructure is often more complex than the retron for a group-oriented transform.

2 STARTP is a marker for PATRAN, the offline program that parses the patterns and creates the data structures that LHASA reads during execution. STARTP marks the beginning of the pattern. The lines from STARTP to ENDP, inclusive, must not be indented.

3 The pattern follows STARTP and may itself include alternative paths information. Detailed instructions for writing patterns are included in the section, “Writing 1-D Patterns for LHASA”.

4 ENDP is the marker to PATRAN for the end of the pattern.

5 The Special Sets available for PCHEM transforms are:

DISCONNECTIVE FGA*NOT*ALLOWED LOW*UTILITY MECHANISTIC RECONNECTIVE STEREOSELECTIVE SUBGOAL(S)*ALLOWED UNMASKING

TCx

Tactical combination knowledge-base tables (TCx) are not subdivided into path lengths. These "transforms" do not actually generate precursors, but are used to guide the selection of sequences of goal and subgoal transforms to perform major simplifications of the target.

Tactical combination transforms in each TCx table are numbered according to the guidelines at the top of that table. A typical TCx transform header looks like:

```
TRANSFORM 5247
NAME Tactical Combination 5247      (Note 1)
...JACS 105, 7352 (1983)
...
...
...      O                  O                   O
...      "                  "                   "
...      C                  C         O         C       O
...     /   \              / \        "        /  \      "
...    C     C=C      =>  C   CH2  C    =>  C   CH    C          (Note 2)
...    |     |   \         |    |       |       |    "     |
...   (C)n-C  (C)n      (C)n –C    (C)n     (C)n-C    (C)n
...          \ /                 \   /                 /
...           C                   C                 C-X
...
      RATING 30       (Note 3)
```

```
...
...      O         @3          O  H      O          O       X  O
...      "         "           "  |      "          "       |  "
...   C-C-C-C-C C-C => C-C-C-C-C C-C => C-C-C=C C C-C =><
...                              |                 |
...                              H                 H          (Note 4)
...
```

Special Sets (Note 5)
...
Target Qualifiers (Note 6)
.... (Note 7)
....

TCx Notes

1 Titles for tactical combinations are simply the string "Tactical Combination" followed by the TC number. The changes effected by most TC's are too complex to be described adequately in a few words.

2 A descriptive picture of the transformations caused by the TC is included since the 2-D patterns actually read by LHASA are frequently somewhat confusing for the chemist.

3 Ratings and rating modifications are not used in evaluating TC knowledge-base entries; a default initial rating of 30 is used.

4 The 2-D pattern sequence for a tactical combination allows LHASA to select transforms to perform the actual modifications to the target structure. Rules for writing 2-D patterns are given in the section, “Writing 2-D Patterns for Lhasa”.

5 The Special Sets available for TC transforms are:

DISCONNECTIVE EXTENDED*SUBGOAL NON*OPPORTUNISTIC PATH*INDEPENDENT REARRANGEMENT RING*REQUIRED STEREOSELECTIVE

Stereochemical (*STEREO) cross-referencing sets may be included in TC transforms, but BROKEN*BONDS information is obtained from the 2-D pattern changes and thus BROKEN*BOND sets are not supported.

6 Target kill qualifiers may be used to eliminate the TC from consideration by LHASA before transforms are attempted to actually perform the constituent retrosynthetic steps.

7 Two sets of four dots (....) must be included even though no mechanism is actually performed for a TC transform.

Long-Range Search Tables

As mentioned previously, the LHASA knowledge base falls into two main sections, the transforms and the long-range searches. Each long-range search is typically devoted to a particularly powerful simplifying transform, e.g. the Diels-Alder or the Birch Reduction. The keying substructure for most of the long-range searches in LHASA is a ring, and for this reason a ring executive (RNEXC) has been included in the Fortran program to handle the appropriate CHMTRN knowledge-base tables. There are currently six of these "ring tables": DZOTB, HALAC, ROBIN, DLSTB, QDATB, and BIRCH.

These tables vary somewhat in format. The older tables, DZOTB and DLSTB, are mostly structured around a binary search type of implementation, with subroutine calls used to check functionality and appendages at various sites around the ring path and to remove obstacles to the eventual simplifying disconnections. The newer tables, HALAC, ROBIN, QDATB, and BIRCH, all use a technique called Prior Procedure Evaluation to pre-rank sequences of subgoal steps leading back retrosynthetically to one of a set of "structure goal", or "S-goal", structure types. Each S-goal type has associated with it a "procedure" in the search table, consisting of a series of calls to "chemistry subroutines" which request the necessary subgoals to convert the target to the S-goal.

One common feature of the long-range search tables is that they consider every possible orientation of the ring that forms their keying substructure. Again, the methods of ensuring this completeness are different in the older tables than in the newer ones, but the central idea is that of "reorienting" the ring substructure. In HALAC, ROBIN, and BIRCH, all reorients are handled within the search table itself, whereas the other three tables give RNEXC the responsibility for performing ring reorientations.

To communicate the keying substructure to RNEXC and to define the patterns for the reorientations of the keying ring(s), a block of ring parameter information is included at the top of each table. This information, which must be the first non-comment lines in the table, is delineated by the keywords RING*PARAMETERS and RING*PARAMETERS*END. For example, in the Quinone Diels-Alder table, QDATB:

```
RING*PARAMETERS
FOR*EACH*ATOM NUMBER*OF*RINGS 2 NUMBER*OF*REORIENTS 1
RING*DEFINITION          MAXIMUM*SIZE 6     MINIMUM*SIZE 6
    ATOM*NUMBERING       1 2 3 4 5 6
    BOND*NUMBERING       1 2 3 4 5 6
RING*DEFINITION          MAXIMUM*SIZE 6     MINIMUM*SIZE 6
    ATOM*NUMBERING       5 6 7 8 9 10
    BOND*NUMBERING       5 7 8 9 10 11
REORIENT*DEFINITION      ATOM*NUMBERING     7 8 9 10 5 6 1 2 3 4
    BOND*NUMBERING       8 9 10 11 5 7 6 1 2 3 4
RING*PARAMETERS*END
```

FOR*EACH*ATOM guarantees that each atom in the ring substructure will be considered as ATOM*1 of the path.

NUMBER*OF*RINGS defines the number of rings in the keying substructure, and NUMBER*OF*REORIENTS specifies the number of reorient definitions that will follow. Keep in mind that this parameter is not the number of orientations that will be considered for each ring. For example, DLSTB has NUMBER*OF*REORIENTS 1, since one reorient definition (see below) is sufficient to set the pattern for reorienting the ring the five times that are necessary. Note also that since HALAC, ROBIN, and BIRCH handle their own reorientations,

NUMBER*OF*REORIENTS is 0 for these tables and no reorient definitions need to be included in the RING*PARAMETERS section.

The RING*DEFINITION defines first the maximum and minimum size of the ring, followed by the numbering systems for atoms and bonds.

Finally, the REORIENT*DEFINITION defines the atom and bond numberings after each reorient. Thus in this example:

<.figure 10>

would reorient to:

<.figure 10>

Within the long-range search tables, some words have a different meaning (or even no meaning) from their definitions in transform tables. For example, END*TRANSFORM is meaningless in a long-range search, in the same way that REORIENT is meaningless in a transform. A BRANCH must end with FINISHED, which, unlike the END*TRANSFORM used in group chemistry tables for this purpose, does not cause a PERFORM THE MECHANISM or a DISPLAY THE PRECURSOR – either of these must be explicitly requested.

REJECT is used only by the long-range search tables, as is CONDITIONALLY*AFTER. Details of the uses of these specifiers may be found in the CHMTRN Manual.

The four tables ROBIN, HALAC, QDATB, and BIRCH use a set of common subroutines that have been gathered together in the file CHEMSR.SRC. This file is automatically appended to each of the main tables before they are compiled by the CHMTRN program.

Writing a long-range search table is a major project. It is advised that anyone considering embarking on such a venture first consult the LHASA group at Harvard for advice and moral support.

Writing 1-D Patterns for LHASA

1-D patterns are used in LHASA for a variety of purposes, most importantly for keying transforms, but also for defining screens, recognizing functional groups, and choosing tactics.

The pattern language used in LHASA allows the definition of the substructures necessary for the above purposes. Associated with each atom and bond in the substructure are the various features that can be used by the pattern writer, either implicitly or explicitly. Implicit features are the features that operate on aspects of the structure or pattern that were not explicitly placed in the pattern or structure by the chemist or pattern writer. For example, LHASA recognizes the presence of hydrogens on atoms even though these have not been drawn in explicitly by the chemist. Similarly the presence of rings of various sizes in patterns is recognized even though the pattern writer may not have mentioned the size of the ring. The features that can be used in patterns include the number of hydrogens on an atom, the atomic charge, aromaticity, etc.

When writing a pattern, it should be remembered that the purpose of the pattern is, in general, to key transforms and not to allow a restricted way into a mass of CHMTRN statements! A considerable amount of work has been performed on speeding up the pattern matching recently and to make the most benefit from this the patterns should be used to the full. When it is decided that a transform should be keyed by a pattern (as opposed to a functional group or pair of functional groups) the CHMTRN writer should attempt to put as much information in the pattern as possible and not just the minimum substructure. This serves several purposes:

a) it allows the unambiguous numbering of atoms and bonds, so that large amounts of CHMTRN are not required just to be able to work round to asking one question about an atom that was not in the pattern;
b) it is easier to debug, resulting in quicker and easier writing of transforms;
c) it is more efficient – it is much quicker for LHASA to use a transform with a large pattern and little CHMTRN than a transform with a small pattern and a large amount of CHMTRN;
d) the more atoms and bonds in the pattern, the more likely the screening in LHASA will perform efficiently;
e) most importantly from the user's point of view, it stops the program wasting time developing sometimes complex subgoal pathways for unsuccessful goal transforms that could have been killed at the pattern-matching stage.

Files containing patterns are compiled using the program PATRAN. The compiler parses the patterns and also perceives a certain amount of other information inherent in the pattern. Warning messages are issued about redundancy and error messages about mistakes.

Once a file is compiled it is screened by the program SCRNSGEN. This program matches all the patterns against another specially selected set of patterns so that LHASA does not need to check every single pattern when performing analyses.

In most situations PATRAN and SCRNSGEN are both run by invoking the command procedure PATRAN. The command procedure supervises the selection of the correct screens and also decides whether SCRNSGEN should be run (only if PATRAN produced some output).

The Pattern Language

The following table is a rigorous definition of the LHASA pattern language:

<Pattern>	::= <Atom Node> <<<Bond Arc> <Atom Node>>* | <"(" <<Bond Arc> <Atom Node>>* ")">* >.
<Atom Node>	::= <Atom Set> <Atom Feature Set> ::= "@"(bonded to) <Atom Number>.
<Atom Number>	::= Sequence number of an atom node in the pattern.
<Bond Arc>	::= <Bond Set> <Bond Feature Set>.
<Atom Set>	::= <Atom Type> <<or> <Atom Type>>*.
<Bond Set>	::= <Bond Type> <<or> <Bond Type>>*.
<Atom Type>	::= Atomic symbols | "X" (any atom type).
<Bond Type>	::= "-" (single bond) | "=" (double bond) | "#" (triple bond) | "%" (aromatic bond) | "&" (any bond type) | "+" (no bond).
<Atom Feature Set>	::= <Null> | <Feature Initiator> <Atom Feature> <<and> <Atom Feature>>* <Feature Terminator>.
<Bond Feature Set>	::= <Null> | <Feature Initiator> <Bond Feature> <<and> <Bond Feature>>* <Feature Terminator>.
<Atom Feature>	::= <Ringsize> | <Heteroatoms> | <Hydrogens> | <Electron Pairs> | <Charge> | <Aromaticity> | <Functionality>.
<Bond Feature>	::= <Fusion>.
<Fusion>	::= "FUSION" <Fusion Assignment> <Fusion Value> <<or> <Fusion Value>>*.
<Fusion Assignment>	::= <Not equals> | <Equals>.
<Fusion Value>	::= "DIARYL" | "ALKYLARYL" | "DIALKYL" |

	"TRIALKYL" | "YES" | "NO" | "EITHER".
<Ringsize>	::= "RINGS" <Ring Assignment> <Ring Value> <<or> <Ring Value>>*.
<Ring Assignment>	::= <Greater Than> | <Less Than> | <Equals>
<Ring Value>	::= <Digit> <Digit>* | "YES" | "NO" | "EITHER".
<Heteroatoms>	::= "HETS" <Hetero Assignment> <Hetero Value> <<or> <Hetero Value>>*.
<Hetero Assignment>	::= <Greater Than> | <Less Than> | <Equals>
<Hetero Value>	::= <Digit> <Digit>*.
<Hydrogens>	::= "HS" <Hydrogen Assignment> <Hydrogen Value> <<or> <Hydrogen Value>>*.
<Hydrogen Assignment>	::= <Less Than> | <Greater Than> | <Equals>
<Hydrogen Value>	::= <Digit> <Digit>*.
<Electron Pairs>	::= "EPS" <Electron Assignment> <Electron Value> <<or> <Electron Value>>*.
<Electron Assignment>	::= <Less Than> | <Greater Than> | <Equals>
<Electron Value>	::= <Digit> <Digit>*.
<Charge>	::= "CHARGE =" <Charge Value> <<or> <Charge Value>>*.
<Charge Value>	::= "NEUTRAL" | "CATION" | "ANION" | "RADICAL" | "YES" | "NO" | "EITHER".
<Aromaticity>	::= "ARYL =" <Aromaticity Value>.
<Aromaticity Value>	::= "NO" | "YES" | "EITHER".
<Functionality>	::= <FG Symbol> <Group Name> <<or> <Group Name>>*.
<FG Symbol>	::= "FGS =" | "FGNOT =".
<Group Name>	::= LHASA functional group or generic group names.
<Feature Initiator>	::= "[".
<Feature Terminator>	::= "]".
<Digit>	::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9".
<Equals>	::= "=".
<Not Equals>	::= "#".
<Greater Than>	::= ">".
<Less Than>	::= "<".
<or>	::= ",".
<and>	::= ";".

* is the Klein Starr operator.

The atom and bond numbering is sequential, i.e. the first atom in the pattern is ATOM*1, the second ATOM*2 etc. The same is true for each bond: the first bond is BOND*1, etc. If there is more than one atom, each atom must be separated from the next by a bond. Each bond must be separated from the next bond by an atom.

Atoms

An atom is defined by its atomic symbol or, if any atom type will suffice, the symbol X. At present atom types must be all uppercase. If an atom can be one of several different atom types, these types should be separated by commas. For example:

N,S,O	Nitrogen, or Sulfur, or Oxygen

Bond order

A bond is defined in LHASA by a symbol representing its bond order:

-	Single bond.
=	Double bond.
#	Triple bond.
%	Aromatic bond.
&	Any bond order.
+	No bond necessary.

The plus can be used for patterns that have two or more fragments where the path between them is not fixed. There is obviously little point in combining the plus with any other bond symbol, but if this is done the results are not predictable. If a bond can have more than one possible bond order these should be separated by commas. For example:

-,=,#	Single, or double, or triple bond.

Examples

A simple linear chain for an alpha,beta unsaturated ketone would therefore be written as follows:

C=C-C=O

It should be obvious however that an unsaturated ester or aldehyde (or a whole host of other species) would also fit the pattern.

In order to signify a branch in a pattern, parentheses are used, an open parenthesis to denote the start of the branch and a close parenthesis to denote the end of the branch. The open parenthesis should follow the atom to which the branch is attached, and the close parenthesis should follow the last atom in the branch. The pattern then continues as if the branch did not exist. Branches may be nested within branches. An example of a branch is:

C=C-C(=O)-C

Note: this could also be written thus:

C=C-C(-C)=O

Implicit Rings

Rings may be defined in a pattern by the @n symbol, where "n" is the number of the atom to which the bond is joined. "n" must be a number lower than that of the previous atom. For example, a ring of size six is written thus:

C-C-C-C-C-C-@1

Explicit Features

Associated with each atom or bond are various features (other than atom type and bond order) that may or may not be included. These are as follows:

For atoms:

HS	number of hydrogens on the atom.
EPS	number of electron pairs on the atom.
HETS	number of hetero atoms alpha to the atom.
RING	ringsize of ring containing the atom.
ARYL	whether the atom is aromatic or not.
CHARGE	presence of a charge or radical on the atom.
FGS	presence of a particular functional group is equired.
FGNOT	absence of a particular functional group is required.

For bonds:

FUSION	presence or absence of a type of ring fusion.

If an atom has any explicit features, these are denoted by an open square bracket, feature, close square bracket. Multiple features are separated using a semicolon within the square brackets. All the features have operators associated with them and values for each operator. The feature takes the form "name" "operator" "value", "value".... The following table summarizes the possible operators and values for each feature.

Features, Operators, and Values

Feature	Operators	Feature Values	Default
HS	= < >	0 1 2 3 4	Any
EPS	= < >	0 1 2 3 4	Any
HETS	= < >	0 1 2 3 4	Any
RINGS	= < >	0 3 4 5 6 7 ...	Any
		YES NO EITHER (0 and NO are equivalent)	
ARYL	=	YES NO EITHER	EITHER
CHARGE	=	ANION CATION NEUTRAL RADICAL YES NO EITHER (NEUTRAL is equivalent to NO)	NEUTRAL
FGS	=	YES NO EITHER Any LHASA functional group	Any
FGNOT	=	Any LHASA functional group	Any
FUSION	= #	DIARYL ALKYLARYL DIALKYL TRIALKYL YES NO EITHER (# means not equal)	EITHER

HS (Hydrogens) Feature

An example of a carbon atom that must have one hydrogen and only one hydrogen atom is:

```
C[HS=1]
```

Every atom in the pattern will be matched to an atom in a structure that is drawn into LHASA by the chemist. If there is a hydrogen atom in the pattern, that hydrogen will only be found if it has been drawn into the target explicitly. Because of this feature it is better to use the HS feature on the atom to which the hydrogen atom would be attached and not to use explicit hydrogen atoms in the pattern.

The only time an explicit hydrogen in a pattern **may** be better than the HS feature would be in a case where the hydrogen was attached to a stereocenter. LHASA requires that all four atoms around a stereocenter be defined; thus one can guarantee that there will be an explicit hydrogen atom. If LHASA ever recognizes stereocenters without explicit hydrogens, all patterns relying on the present requirement will have to be rewritten.

EPS (Electron Pairs) Feature

The EPS feature can be used in conjunction with the HS feature to fix the valency of an atom that can have two or more valency states. For example, to ensure that a phosphorus atom has a valency of three:

```
P[EPS=1](-C)(-C)-C
```

HETS (Heteroatoms) Feature

The HETS feature includes both heteroatoms in the pattern and those not in the pattern, as does HS. An example of an atom with two or more heteroatom neighbors would be:

```
C[HETS>1]
```

RINGS Feature

The rings feature is used to denote an atom in a ring not fully present in the pattern. For example, if an atom is in a ring of size six, all of whose atoms are in the pattern, and also in a ring of size four or five, the following pattern would be used:

```
C[RINGS=4,5]-C-C-C-C-C-@1
```

ARYL Feature

The ARYL feature is used to denote an aromatic atom. All atoms bearing an aromatic bond are automatically given the aromaticity feature, which is used as follows:

```
C-C[ARYL=YES]
```

CHARGE Feature

CHARGE is used to denote the presence or absence of a charge on an atom. For example:

C-O[CHARGE=ANION]

FGS and FGNOT Features

FGS and FGNOT are mainly used to key subgoals.

In LHASA all multiple bonds and bonds adjacent to heteroatoms (excluding aromatic bonds) are recognized as forming parts of functional groups. Each functional group is considered to have one or more "origins" which must be carbon atoms. The FG and FGNOT features are used on **CARBON** atoms only to define them as functional group origins. Any functional group or generic functional group type recognized by LHASA may be used. The format for this feature is:

C[FGS=KETONE]

or

C[FGNOT=LEAVING]

Note that since only carbon atoms can be functional group origins, a construction like

O,N,C[FGS=ETHER]

is illegal.

The groups that may be used in this feature are listed in Appendix II. Since a carbon atom in LHASA can be the origin of more than one group it may be that both the FGS and the FGNOT feature are necessary on the one atom. If both features are on one atom, care should be taken to ensure that the groups in the FGNOT part of the atom features do not remove all the groups in the FGS part of the atom features. A correct use is for example:

C[FGS=LEAVING;FGNOT=IODIDE]

A list of functional groups may be included by using the comma as an OR operator. For example:

C[FGS=KETONE,ALDEHYDE,ESTER]

This means the carbon atom may be an origin of a ketone, aldehyde, or ester.

C[FGNOT=CHLORIDE,BROMIDE]

This means the carbon atom cannot be the origin of a chloride or the origin of a bromide.

For transforms, atom-oriented functional groups and bond-oriented functional groups should not be mixed on the same atom with the same feature. If it is found necessary to have both atom- and bond-oriented groups on the same atom, either two patterns should be allowed to key the transform or KILL qualifiers should be used.

FUSION Feature

The FUSION feature is a feature that is used as a bond modifier. For example, if a bond should be at the fusion of two aromatic rings, the format of the feature is:

C%[FUSION=DIARYL]C

If the bond cannot be at the fusion of an alkyl ring and an aromatic ring the format would be:

C-[FUSION#ALKYLARYL]

A list of fusion types may be given, for example:

C-[FUSION=DIALKYL,TRIALKYL]

Note that the FUSION feature is ignored for ring fusions between rings that are in the pattern themselves.

Multiple Features

If more than one feature is to be used on an atom then the features should be separated by a semicolon. For example:

C[HS=0;HETS=0]

or

C[HS=1;FGS=ALCOHOL,ETHER;RINGS=6,7;HETS>0]

Long Lines

When large patterns are written, inevitably these are too long to place on a single line in a file. The limit for the length of a line in a pattern is eighty characters. Patterns may therefore be split over more than one line by breaking the pattern after a bond, comma, or semicolon. For example:

```
...C[FGS=LEAVING]-C[ARYL=NO;RINGS=YES;
...HS=0,
...1]-C-
...C-C
```

Pattern Symmetry

No automatic account is taken of symmetry within the pattern; this must be done by the pattern writer. Symmetry in the pattern need only be considered if the pattern is symmetrical and the

transform mechanism is symmetrical. A situation where the symmetry in the pattern would have to be recognized by the writer would be in the Paal-Knorr synthesis of pyrroles:

<Figure 12>

```
1   2   3   4   5
N%C%C%C%C%@1
{1,5,4,3,2}
```

In this case the transform would generate two identical precursors even if the appendages on atoms 2, 3, 4, and 5 were different, were it not for the alternative pattern. But in the following intramolecular addition to an enyne it should be obvious that an alternative pattern must not be inserted:

<.figure 12>

```
1   2   3   4   5
O%C%C%C%C%@1
```

Once the pattern writer decides that alternative paths are required, the number of paths and the order in which the atoms appear in those paths must be found. First the writer must work out all the possible symmetrical atom numberings for the pattern. For example:

For the pattern:

```
1  2 3  4   5  6
C-C-C(-C)(-C)-C
```

The symmetrical numberings are:

```
1 2 3 4 6 5
1 2 3 5 4 6
1 2 3 5 6 4
1 2 3 6 4 5
1 2 3 6 5 4
```

The symmetrical numberings are placed in the pattern as a set of alternative paths enclosed in braces as shown below:

```
...STARTP
...C-C-C(-C)(-C)-C
...{1,2,3,4,6,5}{1,2,3,5,4,6}{1,2,3,5,6,4}{1,2,3,6,4,5}
...{1,2,3,6,5,4}
...ENDP
```

Only those alternative numberings that will not affect the mechanism should be included. In some cases it is possible to have more alternative numbering systems than those which also keep a symmetrical mechanism.

Long lines that contain more alternative paths than will fit on one line may be broken after a complete alternative path. The remaining paths may be continued immediately on the next line.

Ratings

It is possible to include a rating in the pattern, although at present this is not used (except in screening). To include a rating, use the following format:

```
...STARTP
...C-C-C
...RATING -30
...ENDP
```

The rating used in screening indicates that the pattern is both a substructure screen and a tactic screen.

Creating Patterns

Transforms are keyed by matching patterns to substructures in the target. The efficiency of the pattern matching depends on how many atoms in the structure there are to be tried for each atom in the pattern. Since the method used is a backtrack search, there is a large overhead if there are a lot of atoms on which to start the pattern matching backtrack phase. It is therefore of great benefit to ensure that the least likely atom is the first atom in the pattern, or that as many of the more unlikely atoms are near the start of the pattern as possible.

To create a pattern, first draw the corresponding substructure and then number it starting from the most unlikely atom, or starting so that as many of the least likely atoms have the lowest possible numbers. Next, write the pattern with the same numbering:

<figure 12>

```
1   2  3                    4   5   6
N%C(-C[FGS=ESTER])%C%C%C%@1
```

Once the pattern has been created, there are several other formatting guidelines to follow. There should be at least three lines before the word TRANSFORM (otherwise PATRAN will not find the first pattern), and at least one line between the line with TRANSFORM and the line with ...STARTP (this is usually taken up by the NAME and references). There should be at least three lines after the ENDP and before the PATHEND, which should be the last or second to last line in the table.

Once the pattern writer is satisfied with the patterns, the patterns should be compiled using PATRAN. Any errors in the patterns will be found and shown, and if the compilation is successful, output for use with LHASA will be produced. Any errors in patterns will have to be corrected before any suitable output for LHASA is produced.

PATRAN also outputs warning messages to indicate what are usually logic errors by the pattern writer. For example:

```
C[RINGS=5,6]-C-C-C-C-C-@1
```

In this case PATRAN would give a warning because the feature on the first atom is redundant (the first atom must be in a ring of size six anyway). No action needs to be taken by the pattern writer, as the feature will usually be ignored. It is likely that any warning messages arise from oversights on the side of the pattern writer and therefore should be considered carefully.

Automatic Screening of Patterns

Once the patterns have been successfully compiled using PATRAN, screening will take place automatically using SCRNSGEN. The final output from this process is in the form of two files, one for the pattern writer to read and check and one for LHASA to use. The file for the pattern writer to read will be of type .LST and should be checked to ensure that the correct screens have matched. For instance, if the pattern contained an indole, the indole screen would have to match to that pattern.

A copy of all the screens is kept in the file LHC:SBSCRNS.SRC, and this can be used by the pattern writer to see which screens would be expected to match. Once any obvious corrections have been made, and the screens are what the pattern writer expects, they can be tested.

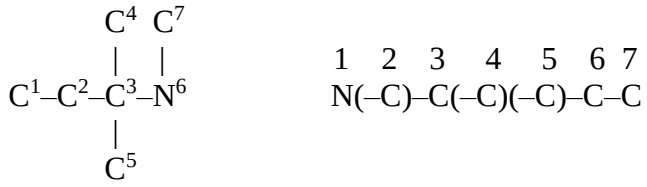
The file containing the screens for functional group recognition is different from other screening files. The file used for functional group screening is LHC:FGSCRNS.SRC.

The Pattern Matching Process

The pattern matching in LHASA uses two principles – Set Reduction and Backtrack Searching. Initially a set of candidate atoms is made up for each pattern atom and a set of bonds for each pattern bond. A count is made to see how many atoms have identical properties. If any of the sets have insufficient candidates then pattern matching is immediately abandoned as a match will not be possible.

Once a set for each atom and bond has been created a backtracking search begins. The first atom in a set is taken, then the first atom in the next set which is also ALPHA to the previous selected atom is tried. If there are no candidate atoms then a backtrack occurs and the previous atom is abandoned, the selection of the next atom in that set being taken. When a candidate is found a check is made to see that the bond between the two atoms is correct. Assuming it is, matching continues. When all the possibilities have been tried matching is abandoned.

This is best illustrated by an example:



Structure Pattern

The set reduction would appear something like:

Set Reduction for pattern atom 1: 6
Set Reduction for pattern atom 2: 1 2 3 4 5 7
Set Reduction for pattern atom 3: 1 2 3 4 5 7
Set Reduction for pattern atom 4: 1 2 3 4 5 7
Set Reduction for pattern atom 5: 1 2 3 4 5 7
Set Reduction for pattern atom 6: 1 2 3 4 5 7
Set Reduction for pattern atom 7: 1 2 3 4 5 7

The backtrack searching would look something like:

Pattern atom 1 trying atom 6
Pattern atom 2 trying atom 3

```
Pattern atom   3  trying atom   7
Pattern atom   4  trying atom   0
Pattern atom   3  trying atom   0
Pattern atom   2  trying atom   7
Pattern atom   3  trying atom   3
Pattern atom   4  trying atom   2
Pattern atom   5  trying atom   4
Pattern atom   6  trying atom   5
Pattern atom   7  trying atom   0
Pattern atom   6  trying atom   0
Pattern atom   5  trying atom   5
Pattern atom   6  trying atom   4
Pattern atom   7  trying atom   0
Pattern atom   6  trying atom   0
Pattern atom   5  trying atom   0
Pattern atom   4  trying atom   4
Pattern atom   5  trying atom   2
Pattern atom   6  trying atom   5
Pattern atom   7  trying atom   0
Pattern atom   6  trying atom   0
Pattern atom   5  trying atom   5
Pattern atom   6  trying atom   2
Pattern atom   7  trying atom   1
MATCH = .TRUE.
Matched Target Atoms:   6  7  3  4  5  2  1
```

The bonds between the atoms are never indicated since there can only ever be one bond between two adjacent atoms. If the bond between the two atoms is however incorrect, then there will be a backtrack which will appear to be retrying an already matched atom, for example:

```
Pattern atom   4  trying atom   3
Pattern atom   4  trying atom   0
```

When the backtracking has been exhausted LHASA moves on to try the next available pattern.

Where there is more than one way of matching a pattern onto a given structure LHASA will find all the possible matches if required (as is the case in transform matching). Full account is taken of symmetry in the target structure however, so a transform will not be seen acting twice on a structure due to symmetry within that structure.

Subgoals

The set reduction and backtrack search discussed so far is used as described only for pattern matching not requiring subgoals. When transforms are applied at subgoal level in LHASA the set reduction and backtrack search work in a similar way but with some visible differences. Assuming the transform is allowed to request subgoals, that is it contains the SUBGOAL(S)*ALLOWED directory set, the sequence of events to test a given transform with subgoals would be as follows:

At the first pass through an attempt to match the pattern in the normal manner would be made. If the pattern matched then the transform would be applied in the usual manner.

If the pattern failed to match exactly, the reason for failure would be examined. If the reason was a missing feature that could not be corrected by subgoals the pattern would be removed from further testing. If the transform had fewer FGS features in the pattern than the next level of pattern matching required (at each subgoal level in pattern-keyed chemistry LHASA is able to rectify one more FGS mismatch than at the previous level) the pattern would be removed from further testing.

At each subgoal level the pattern would be subjected to the same set reduction as before. This time, however, if a set were found to contain insufficient atoms the pattern matching would not stop, but would continue knowing that subgoals would have to be applied. If the total number of subgoals found to be needed whilst doing set reduction exceeds the current level of subgoals, pattern matching terminates. Once all the set reduction has been performed the backtrack search begins. Whilst performing the backtrack search a record is kept showing which atoms need to be corrected by subgoals.

The debug obtained from attempting to match a pattern whilst performing subgoals looks different to the debug normally seen in two respects. The first difference is that carbon atoms can be matched to a carbon atom in the pattern even if the functional group necessary for the pattern to match is not correct. The second difference is that for terminal carbon atoms in the pattern the pattern matching may proceed even if the atom does not exist. The first of these two features will be seen for carbon atoms that possess the FGS or FGNOT feature. The second will be seen only for terminal carbon atoms that possess the FGS feature. All non-carbon atoms in the pattern, and the bonds to them, must match the target structure exactly. Subgoal requests can only be generated by FGS and FGNOT features and by bond-order mismatches between pairs of carbon atoms.

Debugging of Transform Patterns

This section is intended for those writing patterns to key transforms. The principles are the same for all patterns, though the debug switches may vary.

If the pattern has been written as part of a transform, then run LHASA in test mode, selecting the correct transform number and turning on PATTERN debug. If the pattern matches as expected then there is no debugging to be done. If the pattern does not match then the reason it failed needs to be investigated. Note that the debug file should be checked to see if the pattern matched, as the matching of the pattern alone does not determine whether the transform will be seen or not.

Assuming the pattern did not match, the first thing to check is whether it was screened out by the substructure or tactic screening before pattern matching was attempted. Carefully check to see whether the pattern was screened out by the substructure screen. If it was, then the pattern is probably incorrect. Go back and check which screens matched and which did not in order to determine why the transform was screened out. If the pattern was screened out by TACTIC screening then the writer should correct the problem by either selecting the correct tactics or by altering the directory sets at the top of the appropriate transform.

If the pattern was not screened out but failed to match, further debugging information will be needed to locate the problem. Run LHASA again, this time selecting PATTERN and PERCEPTION debug.

The first type of pattern-matching failure, a problem in the set reduction, will show up in the debug file as one of two possibilities. If the debug file shows the set reduction ceased and that there are no atoms (or bonds) with the correct features (as indicated by a set for pattern atom(bond) x with NONE in it), then the structure did not have the necessary atoms (or bonds) with the correct features to achieve a match. Assuming the writer has drawn in the correct structure, a check then needs to be made to ensure that all the features put into the pattern are present in the structure drawn in. In the debug file the atom (or bond) that failed set reduction is

the last one mentioned, so the writer needs to ensure that there are enough atoms (or bonds) in the structure with a given set of properties to satisfy the pattern. The main things to check for are features that the program may have recognized and that the pattern writer may have overlooked, for instance aromaticity or charge.

Once the problems of set reduction have been overcome, and if the pattern still does not match, the problem must be in the backtracking process. Usually, you will find that the pattern atoms are joined in the wrong order, especially where branches occur. To solve this type of problem, write down the structure with its numbering and write down the structure numbers that should match each pattern atom for a successful match. Check through the debug file until the failing atom (or bond) is found.

The debugging of patterns whilst performing subgoals becomes slightly more complex. If PATTERN debug is turned on, the pattern should match with certain mismatches indicated. LHASA tries to correct any mismatches via subgoal transforms before attempting the main transform. Before attempting the main transform and after attempting the subgoals LHASA makes a last check to see that the structure now being used has the correct features. If the structure does not have the correct features because subgoals did not correct the problem, the transform is abandoned and a warning is issued to the user.

Defaults for Patterns

The defaults for the various features may be changed by including appropriate command lines in the file of patterns. For example, to change the default for ARYL (normally EITHER) one would include the following lines:

```
...DEFAULTS
...ARYL=NO
...ENDD
```

As many feature defaults can be changed as wanted as long as each default is placed on a separate line. For example:

```
...DEFAULTS
...ARYL=NO
...HS=2,3
...FUSION#DIARYL
...ENDD

    .

    .

    .

...DEFAULTS
...FUSION=ANY
...ENDD
```

The defaults set up will remain until reset by another series of commands. There should be three blank lines before and three blank lines after the section that sets the defaults. Defaults cannot be set from within a pattern, that is, not between the TRANSFORM and ...ENDP lines.

It is also possible to define generic atom type defaults. The symbols Z1 to Z20 are used for this purpose. Generic atom symbols may be used instead of the atomic symbols and are defined along with any other defaults. For example:

```
...DEFAULTS
...Z1=N,S,O
...ARYL=YES
...ENDD

    .

    .

    .

      TRANSFORM 1000
      NAME Test
...STARTP
...Z1%C%C%C%C%@1
...ENDP
```

If a generic atom symbol is used that has not been defined, it is assumed to be equivalent to X (X = any atom type).

Specific Applications of Patterns in LHASA

The patterns described so far are intended for use in transforms. This section describes restrictions when writing patterns for other purposes, such as screening and functional group recognition. The main restrictions are imposed by the FGS and FGNOT features.

Patterns for Screening

Patterns for use as screens in pattern chemistry must not include any atom or bond that could be modified by a subgoal. This means that the features FGS, FGNOT, HS, EPS, HETS, CHARGE and BOND ORDER (except aromatic) cannot be used in screens. The features that can be used for screens are RINGS, ARYL, FUSION and ATOM TYPE. No negative feature can be used in screens. For example, RINGS=NO cannot be used.

The screens are screened against themselves, so efficient ordering is very important. Any screen that can match a screen that occurred before it in the file is in the wrong place! All files using patterns must be recompiled if any screens are changed or added.

Patterns Functional Group Recognition

Patterns to be used for functional group recognition purposes can use all the features, but care must be taken with FGS and FGNOT. These two features can only be used provided the groups they refer to precede the patterns using them in the file. A separate set of screens exists for functional groups. These screens must **not** be altered.

There are in addition several special features that are used only for functional group patterns. These features are used to tell the mechanism processing routine (MCHRDR) how to construct functional groups in response to CHMTRN ATTACH and INTRODUCE mechanism commands. ORG1 labels the first origin (CARBONx*1) of a multi-origin group. ORG2R labels the point of attachment for the second locant in a two-centered ATTACH or INTRODUCE

command. The atom labeled ORG2R is not itself attached. ORG2A also labels the point of attachment for the second locant in a two-centered ATTACH or INTRODUCE command; however, ORG2A atoms are themselves added to the new functional group before it is attached to the structure. HETORG labels the corresponding atom in the structure as the HETERO*ORIGIN (see the CHMTRN Manual). EXTRAMETHYLS=n labels an atom that needs n extra carbon atoms attached to it in the mechanism.

Certain restrictions apply to the use of these special features. Only one atom can be labeled ORG1. Similarly, there can be only one ORG2x and only one HETORG. The ORG1 atom can also be labeled ORG2A, but not ORG2R. EXTRAMETHYLS cannot be used on the ORG1 atom or the ORG2R atom. ORG2R, ORG2A, and HETORG are all optional. ORG1 is also optional, but its absence signals a functional group that cannot be legally ATTACHed or INTRODUCEd in a mechanism command. If there is no ORG2x feature, the group cannot be used legally in a two-centered mechanism command. Finally, if a functional group is described by more than one pattern, only one of the patterns may include mechanism features, and this is the pattern that will be used for mechanism commands.

Multiple Patterns for Transforms

It is possible to associate more than one pattern with a single transform. The use of multiple patterns is achieved in the following way:

```
...STARTP
...C(-C)%C%C%C(-C)%C%C%C@1
...STARTP
...C(-C)%C(-C)%C%C%C%C%@1
...ENDP
```

In the above example the benzene can be either ortho or para substituted.

Patterns for Tautomer Recognition

When writing patterns for use in tautomer recognition, the pattern(s) included must be the uncommon one(s), the one(s) that do(es) not show up in the transform knowledge base. If both tautomers appear in the knowledge base, two tautomer transforms are necessary.

Multiple uncommon patterns can be used in a tautomer transform provided that the transform mechanism is adequate to convert each uncommon form to the tautomer that appears in the transform knowledge base. There is no longer any restriction on the features that can be used in tautomer patterns.

Patterns for Strategy Transforms

Patterns for Strategy transforms have the identical rules as those for ordinary transforms.

Elements of Pattern-writing Style

Variations of style within patterns are very limited; however, there are some that can be applied.

The use of defaults should be restricted so as to minimize confusion for anybody looking at a specific transform in the table. There is not much point, for example, in setting the default for HS to zero if there is only one transform in the table that will benefit.

Spaces within a pattern should not normally be needed, but if the pattern uses a lot of features on the atoms and bonds, separating each atom and bond from the rest using a space makes things a lot clearer. If the number of qualifiers per atom is particularly large, it is often

best to place each atom on its own line so that each line does not have to be searched for pattern atoms.

All letters in a pattern must be uppercase. This is required by PATRAN and cannot be changed.

Since PATRAN checks only the first ten characters of any functional group name the extra letters can be left out with no effect on the pattern. In general, the full name of any functional group should be placed in the pattern unless there is some other serious constraint. It will be a lot easier for people in future to read the full functional group specification rather than try to work out the abbreviated form.

The best place to break long lines in a pattern is after a bond. If possible, the bond should have no features associated with it other than bond order.

Writing 2-D Patterns for LHASA

2-D patterns are used in LHASA for keying tactical combinations (TCs) of transforms, for selecting transforms from the knowledge base to be used as subgoals or as candidates for a TC step, and for checking group-oriented transform keying in certain transform tables.

The 2-D pattern language allows the depiction of keying substructures and of substructures undergoing change as sequences of atoms bonded in two dimensions. Each atom on the "main line" of the path can have up to four substituents; left, right, above, and below. Valencies not explicitly specified can map to any target structure atom. Since these patterns describe the changes effected by a transform, both a retron and a precursor substructure are needed in each transform pattern. Several examples of 2-D patterns are included in the sample transform headers in the subsection, “Individual Transform Tables”, and in most of the transform tables in the core knowledge base.

When writing a pattern, it should be remembered that one purpose of the pattern is to key transforms or TCs. As such, the keying element should be specified as completely as possible without compromising generality. Performing the pattern mapping operation on a simple substructure, e.g., a string of several carbon atoms, is time consuming as there will be many mappings. However, if there are functional groups in the pattern or if the atoms are constrained to a ring, the mapping operation will be greatly accelerated. Files containing patterns are compiled using the program PARSE, which is invoked by the CHMTRN compiler. PARSE parses the patterns and also perceives functional groups and calculates change codes describing the differences between retron and precursor patterns. Output from PARSE is written into a .PRS file for each CHMTRN table containing 2-D patterns. Specifying the argument PLIST to the CHMTRN procedure causes the generation of a .LIS file which details the results of the pattern parsing.

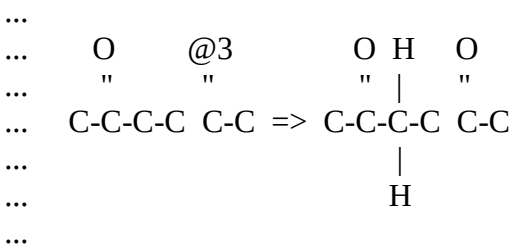
The 2-D Pattern Language

The 2-D Pattern Path

Reactions are represented retrosynthetically, with the retron pattern on the left and the precursor pattern on the right, separated by an arrow (=>). The path of a 2-D pattern consists of the atoms on the "main line", or the line containing arrows. There must be a 1:1 correspondence between the path carbon atoms in the retron and precursor patterns, and thus the same number of path carbons on either side of the arrow.

In knowledge-base tables which employ 2-D patterns as retron keying elements (currently only TC tables), path atom and bond numbering is sequential, i.e. the first path atom in the pattern is ATOM*1, the second ATOM*2 etc. The same is true for each bond: the first bond is BOND*1,

etc. If there are several path atoms, they need not all be joined to each other by path bonds. However, the bond numbering proceeds as if there are path bonds joining all path atoms. For instance, BOND*4 is missing in the retron pattern below, and the bond between ATOM*5 and ATOM*6 is BOND*5.



Atoms

An atom is defined by its atomic symbol in uppercase letters. If an atom can be one of several different atom types, these types should be separated by commas. For example:

C,N,O	Carbon, or Nitrogen, or Oxygen

Onpath hydrogens have been known to cause mapping problems. If possible, hydrogens should be put above or below the path of the pattern.

Superatoms

In addition to the common atom types of the periodic table, there are many clusters of atoms frequently used to depict functional groups in organic chemistry. For instance, the group ESTER is represented by the "superatom" -COOR while an AMIDE*3 can be depicted as -CONR2. Numerous superatoms are supported in the 2-D pattern language. The complete list of superatoms and superatom definitions is included in Appendix I at the end of this Guide.

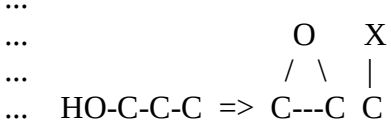
If a superatom can be one of several different types, these types should be separated by commas as for simple atom types.

Bonds

A bond is defined by a symbol representing its bond order:

-	Single bond
=	Double bond
#	Triple bond
%	Aromatic bond

Each onpath atom must have the same number of bonds in the precursor pattern as it did in the retron pattern. Bonds may be extended horizontally (but not vertically) for clarity, as shown in the epoxide below:



Branching must be done above and below the path, as parenthesized appendages are not allowed onpath. For example, the following pattern is illegal:

```
...
...   C-C(-C)-C
...
```

and should be written:

```
...
...      C
...      |
...   C-C-C
...
```

Functional Groups

Functional groups in 2-D patterns may be hung off the main line or included on it. They may be depicted using simple atoms and bonds, or using superatoms.

Explicit Rings

Rings may be defined in a pattern by any bond symbol combined with the @n symbol, where "n" is the number of the atom to which the bond is joined. "n" must be a number lower than that of the previous atom. For example, a ring of size six is depicted as:

```
...
...                                        @1
...                                        |
...   C-C-C-C-C-C-@1     or     C-C-C-C-C-C
...
```

Long Patterns

When long patterns are written for transforms or for TCs, they often will not fit on a single line in a file. The limit for the length of a line in a pattern is eighty characters. Patterns may be split over more than one line, but only by breaking the pattern after a retrosynthetic arrow (=>).

Multiple 2-D Patterns for Transforms

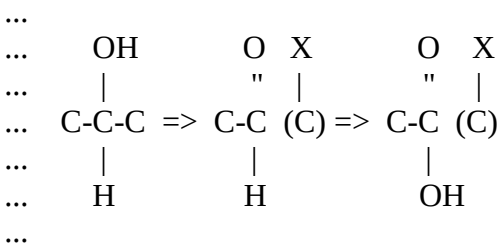
It is possible to associate more than one pattern with a single transform if it is not appropriate to draw a single general pattern. Each pattern must be preceded by its own RATING statement or block of rating specifiers, even if the ratings are identical, in which case the specifier SAME*RATING is used. This multiple pattern capability is not available for tactical combinations.

Special Considerations for Tactical Combinations

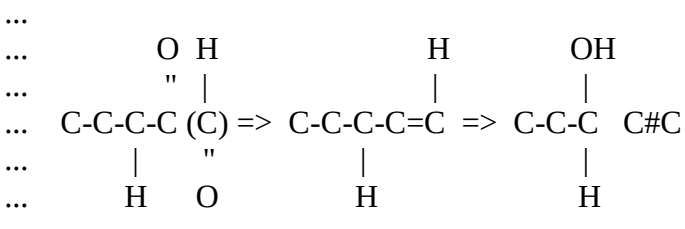
Parenthesized Pattern Fragments

In writing sequences of 2-D patterns for tactical combinations, precursor fragments which will not be carried further in the retrosynthetic analysis should be parenthesized in the precursor pattern.

For example,



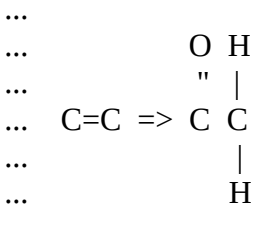
Similarly, pattern fragments that are reconnected to the main fragment(s) should be parenthesized in all the steps up to and including the reconnective step.



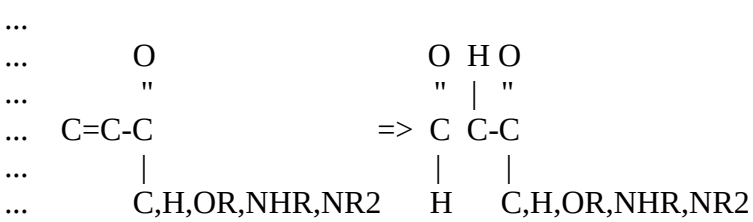
Creating 2-D Patterns

2-D patterns must have the keying retron substructure accurately specified and all carbon atoms involved in bonding changes onpath. To illustrate these points consider the following examples:

1. The atom and bond changes effected by the Aldol transform may be represented as:



However, this pattern fails to specify the withdrawing group necessary for application of the Aldol transform. A complete representation would be:



2. The following 2-D pattern is illegal since it includes changes to an offpath (i.e. above or below the "main line") carbon atom.

```
...
...   CHO               COOH
...    |                 |
...   C-C-C-C-C  =>  C-C-C-C-C
...
```

The pattern should be written:

```
...
...   O                   O
...    "                   "
...   C-C-C-C-C-C  =>  C-C-C-C-C-C
...    |                   |
...   H                   OH
...
```

The 2-D Pattern Mapping Process

The 2-D pattern mapping phase begins with the selection of one target structure atom to map to one of the pattern path atoms. These atoms are usually selected based on functional group correspondence. For instance, if there is a ketone in both the target and the pattern, the ketone origin in the target will be chosen as the "anchor atom" to begin the mapping. The complete mapping is then grown out from this anchor position by considering atoms alpha to the last mapped target atom. For each path atom there may be several target atoms which map. The first candidate is used for the current mapping while the rest are saved for a backtracking phase. After the first mapping is complete, alternative candidates are considered for each path atom, starting from the last path position mapped. To illustrate this process consider the example:

<There is no placeholder reference in the runoff file to an image for the structure below>

```
NR2
 |
C-C-C-C-C-C
```

Structure Pattern

Mappings

1: 2 3 4 5 6 7 Alternative at position 6 = 11
2: 2 3 4 5 6 11 Alternative at position 4 = 9
3: 2 3 4 9 10 11 Alternative at position 1 = 5
4: 5 4 9 10 11 6 Alternative at position 2 = 6
5: 5 6 11 10 9 4 Alternative at position 1 = 8
6: 8 7 6 5 4 3 Alternative at position 6 = 9
7: 8 7 6 5 4 9 Alternative at position 4 = 11
8: 8 7 6 11 10 9 Alternative at position 4 = 11

The above sample underscores the exhaustive nature of the mapping process which ensures that all possible matches are found.

Debugging of 2-D Patterns

Errors in 2-D patterns surface upon parsing the new pattern or when running the transform for which the pattern was written.

If a parsing error is detected, the compilation job should be resubmitted with the PLIST option enabled. The PLIST flag will cause creation of a .LIS file which contains information on the parsing results and detailed error messages.

Alternatively, if the pattern mapping fails during transform or TC execution, then manual selection of the defective transform with the debug button 2-D PATTERN MATCHING enabled will yield information on the attempted mapping in the log file.

2-D Pattern-keyed Subgoals

The details of generating subgoal requests from 2-D pattern mismatches are of a complex, technical nature and do not lend themselves readily to delineation in this manual. However, from a chemical standpoint, this subgoal capability is based on a system of perceiving differences between 2-D patterns and between a target structure and one substructure of a 2-D pattern.

Subgoal processing begins with the identification of a powerfully simplifying "goal" transform or tactical combination that maps only partially to the target structure entered by the chemist. The differences between this target and the retron pattern of the goal TC are then perceived in terms of the atom and bond changes necessary to rectify the mismatch. The resulting strings of atom and bond changes are canonicalized to an integer code (change code) which uniquely describes the overall differences to be rectified by a subgoal transform. Then, since each transform in the knowledge base has retron and precursor 2-D pattern substructures, the changes effected by each transform can be similarly perceived and canonicalized to a change code. Finally, by comparing the change code for the desired subgoal step with codes in the knowledge base, candidate subgoal transforms are identified.

The advantage to this system over traditional, functional-group-based subgoals is that, in principle, any transform in the knowledge base can be used as a subgoal to achieving a goal. Thus, the restraint of using only FGI, FGA, and FGR transforms as subgoals is relaxed.

Coding and Debugging - An Example

This section is a complete example of the coding and debugging of a new transform. The transform was written from a standard LHASA UK Reaction Summary Form. The actual summary form, copies of the various versions of the transform, and copies of debug output are included as figures in the text for your convenience in following along with the process of writing and debugging the transform.

Our treatment of this example is somewhat chronological. We've included a number of early mistakes to illustrate how to correct them. Thus it may be dangerous to take any pieces of this section out of context. One particularly useful comparison, though, should be the first and last versions of the new transform, Figures 11.2 and 11.3.

Chemical Groundwork

A completed Reaction Summary Form for the new transform, Proto-desulfonation of Aryl Sulfonic Acids, was obtained from the chemist (see Figure 11.1) and considered to decide

whether sufficient data had been supplied. The chemist was asked whether or not he really did not want the reaction to apply to anthraquinones (Figure 11.1c). It was decided that they should be allowed, but that desulfonation from the alpha position was not generally activated in these cases.

Generating a Trial Knowledge-base Table

Before getting down to the nitty gritty of writing qualifiers, we had to decide where the transform would fit into the LHASA transform knowledge base. Referring back to the section, "Here's My Transform - Where Do I Put It?", we found that the desulfonation was clearly a path change 0 addition of functionality on an unfunctionalized center and should therefore go into the FGATB0 functional group addition table.

Next, a transform number for the new transform had to be chosen. Searching LHXL:TRNSFM.DAT for the string "FGATBx" revealed that the lowest number ("Reserved for future FGATBx transform") was 720.

Name **Date**

Lab.No. **Phone**

Reaction Name (e.g.Michael Addition)

Short Description (complete if name not satisfactory)

Stereospecific? Yes No **Regiospecific?** Yes No

References (include a review if possible)

Company Secret? Yes No

Not all of the following sections will be relevant in all cases. Please fill out all you can, attaching further sheets if necessary.

Figure 11.1a Completed Reaction Summary Form

Generalized Reaction Description

Please draw in a retrosynthetic sense the smallest substructure that is essential, or unavoidable, for the reaction. Complete all valencies with numbered groups. The following abbreviations may be useful: W = withdrawing, D = donating, X = leaving group. Please number the product, starting with any convenient atom as number 1, numbering essential or unavoidable appendages as you come to them. Then number the starting material(s), using the product numbering for corresponding atoms. Finally, for pattern chemistry transforms, write out a draft pattern.

<.figure 30>

Figure 11.1b Completed Reaction Summary Form

Structural Requirements of Starting Materials

How would various groups near or adjacent to the reaction site help or hinder the reaction? Would some combinations completely prevent the reaction from occurring? Consider withdrawing, donating, and leaving groups as well as steric bulk. If possible, tabulate and include yield ranges. Consider bond by bond whether any bond can or cannot be cyclic, acyclic, or aromatic. If a bond is a ring bond, does this factor have an effect on the yield? Refer to bonds by atom numbers from the previous page.

<.figure 40>

Figure 11.1c Completed Reaction Summary Form

Reaction Conditions

A. Reagents, solvents, pH, temperature, etc.

<.figure 10>

B. Work-up

<.figure 10>

Initial Rating for Transform

Note: Refer to Chapter 9 of Transform Writers' Guide for definitions of the following scales:

	Bad	Poor	Fair	Good	V.G.	Excellent
Typical yield	[]	[]	[]	[]	[]	[]
Reliability	[]	[]	[]	[]	[]	[]
Reputation	[]	[]	[]	[]	[]	[]
Homoselectivity	[]	[]	[]	[]	[]	[]
Heteroselectivity	[]	[]	[]	[]	[]	[]
Orientational selectivity	[]	[]	[]	[]	[]	[]
Condition flexibility	[]	[]	[]	[]	[]	[]
Thermodynamics	[]	[]	[]	[]	[]	[]

<figure 10>

Figure 11.1d Completed Reaction Summary Form

Test Examples

Please provide at least three good test cases, with yields if possible. These may be to either succeed or fail.

<.figure 45>

Figure 11.1e Completed Reaction Summary Form

Writing the Transform

We now set about writing our new transform, typing it into a dummy FGATB0.SRC file containing no other transforms. The first few lines were easy – we just followed the sample transform at the beginning of the section, "What Does a Transform Look Like?", and filled in the transform number, name, references, pattern, authorship, and path change.

Fortunately, our participating chemist had done the hard work of filling out the rating scales on the Reaction Summary Form, and our job was simply to put the appropriate keywords into the transform entry. The offline CALCRAT program was useful in generating the correct block of text.

Next, we had to choose a GROUP*1 for the transform. In the FGATBx tables, of course, this is the group to be added by the transform. Consulting the CHMTRN Manual, we found C*SULFONATE, which looked almost like a sulfonic acid, and decided to use that LHASA functional group type as GROUP*1.

The last easy step in writing the new transform was to choose the special set labels that had to be included in the transform entry for cross-referencing purposes. Consulting the list of special sets for FGATBx tables in the subsection, "FGATBx", we found that the available labels were LAST*RESORT and STUDENT. Neither of these labels was appropriate for our new transform, so we did not include any special set labels. Also, the transform does not break any bonds (except C-H) and does not affect stereochemistry, so <BROKEN*BONDS and XXXXX*STEREO sets were likewise not included.

Having dispensed with the preliminary information for the transform, we were ready to start writing qualifiers. At this stage we went back to the Reaction Summary Form and systematized the information provided by the chemist. This step made it much easier to write the actual qualifiers. The following generalizations seemed appropriate:

1 The transform must only be applied to generate aromatic sulfonic acids.

2 We should add to the rating for donating groups on the ring, incrementing more at the ortho positions than at the para position than at the meta positions.

3 We should subtract from the rating for withdrawing groups on the ring, decrementing more at the para position than at the meta positions.

4 We should add to the rating for withdrawing and any other types of groups at the ortho positions, because of the release of steric strain.

5 We should allow target structures that are fused ortho/meta or meta/para, but should add to the rating in the first case and/or subtract in the second.

6 We should allow anthraquinone-type targets, but should not do the fusion increments/decrements mentioned in point 5.

Obviously, the first problem in writing the transform was going to be figuring out how to isolate the atoms on the aromatic ring that are ortho, meta, and para to the carbon getting the sulfonic acid in order to ask questions about what kinds of substituents they have on them. The "path" for a Path Change 0 functional group addition transform is as short as it can be, i.e. only one atom (ATOM*1), so we had to figure out labels for the rest of the atoms on the ring ourselves. The ortho atoms were easy – they would be simply the atoms ALPHA TO ATOM*1. Meta was

a little harder since there could be as many as four atoms BETA TO ATOM*1, only two of which would be the meta carbons on the aromatic ring. Obviously we needed a way of distinguishing between onring and offring locants. Fortunately, a relatively new CHMTRN feature allowed us to DESIGNATE a ring as the CURRENT*RING, after which we could "qualify" our locants, e.g. BETA TO ATOM*1 ON*CURRENT*RING (see the CHMTRN Manual entries for DESIGNATE and the QLQUAL field). The para atom then followed: it was simply the atom GAMMA TO ATOM*1 ON*CURRENT*RING.

Having settled on this approach for specifying ring locants, we saw that the code for points 1-4 above should be quite straightforward. For point 5, we needed to be able to identify fusion situations around the ring. This turned out not to be much of a problem, since obviously a fusion atom ortho means ortho/meta fusion and a fusion atom para means meta/para fusion.

Finally, we had to be able to identify anthraquinone-type targets. In the interest of generality, we decided to call any target structure with a ketone alpha to an ortho atom and an aromatic atom on the other side of the ketone "anthroquinonish". Generality aside, this decision was taken principally to avoid having to try to identify the full anthraquinone substructure in CHMTRN.

Using these guidelines, our first try at a proto-desulfonation transform looked like the one shown below in Figure 11.2.

```
      TRANSFORM 720
      NAME Proto-desulfonation of Aryl Sulfonic Acids

...Sulfonation and Related Reactions, E.E. Gilbert,
...   Interscience, 1965, pp. 425-442.
...Mechanistic Aspects in Aromatic Sulfonation and
...   Desulfonation, H. Cerfontain, Interscience,
...   1965, pp. 185-195.
...The Organic Chemistry of Sulfur, C.M. Suter,
...   J. Wiley and Sons, 1944, pp. 387-394.
...Methoden der Organischen Chemie, v. 9, 1955, pp. 535-539.
...
      TYPICAL*YIELD          GOOD
      RELIABILITY            FAIR
      REPUTATION             GOOD
      HOMOSELECTIVITY        FAIR
      HETEROSELECTIVITY      FAIR
      ORIENTATIONAL*SELECTIVITY EXCELLENT
      CONDITION*FLEXIBILITY  FAIR
      THERMODYNAMICS         GOOD
...
... H       SO3H
...  |       |
... AR => AR
...
      ...Authors: Tony Baxter (ICI) and Alan Long (Harvard)
      ...Completed: August 1987
       ...PATH CHANGE 0
       GROUP*1 IS C*SULFONATE
       ...
       KILL IF ATOM*1 IS NOT IN AN AROMATIC ATOM
```

```
      KILL IF THERE IS NOT A HYDROGEN ON ATOM*1
      DESIGNATE THE RING CONTAINING ATOM*1 AS THE  &
        CURRENT*RING
...
...   Increment for donating groups on the ring
...
      ADD 15 FOR EACH DONATING GROUP ON ALPHA TO ATOM*1
      ADD 10 IF THERE IS A DONATING GROUP ON GAMMA TO  &
        ATOM*1 ON*CURRENT*RING
      ADD  5 FOR EACH DONATING GROUP ON BETA TO ATOM*1  &
        ON*CURRENT*RING
...
...   Decrement for withdrawing groups on the ring
...
      SUBTRACT 15 IF THERE IS A WITHDRAWING GROUP ON  &
        GAMMA TO ATOM*1 ON*CURRENT*RING
      SUBTRACT 5 FOR EACH WITHDRAWING GROUP ON BETA TO  &
        ATOM*1 ON*CURRENT*RING
...
...   Increment for substituents in the ortho positions for
...     relief of steric strain
...
      ADD 10 FOR EACH NON HYDROGEN ON BETA TO ATOM*1  &
        OFF*CURRENT*RING
...
...   Increment for ortho/meta fused systems, and
...   decrement for meta/para  fused systems, but
...   only if the fused systems are not anthraquinonish.
...
      FOR EACH CARBON BETA TO ATOM*1 ON*CURRENT*RING DO CBETA
        IF SPECIFIED*ATOM 1 IS NOT A FUSION ATOM GO TO CBETA
        IF ALPHA TO SPECIFIED*ATOM 1 IS A KETONE AND:IF  &
          ALPHA TO THE PREVIOUS LOCANT OFF*CURRENT*RING  &
          IS A FUSION ATOM THEN GO TO CBETA
        ADD 25 IF ALPHA TO SPECIFIED*ATOM 1 IS A FUSION  &
          ATOM AND:IF THE PREVIOUS LOCANT IS THE SAME  &
          AS ALPHA TO ATOM*1 ON*CURRENT*RING
        SUBTRACT 20 IF ALPHA TO SPECIFIED ATOM 1 IS A  &
          FUSION ATOM AND:IF THE PREVIOUS LOCANT IS THE  &
          SAME AS GAMMA TO ATOM*1 ON*CURRENT*RING
CBETA  ENDDO
      CONDITIONS pH<1
      CONDITIONS pH:1
      CONDITIONS PH2:4
      CONDITIONS pH4:6
      ....
        ATTACH A C*SULFONATE TO ATOM*1
      ....
```

Figure 11.2 Proto-Desulfonation Transform - First Try

Compiling and Debugging the Transform

First Try

Now that we had written a version of the transform, we were anxious to try it out. First, we had to run our FGATB0.SRC file through the CHMTRN compiler to create a binary file readable by LHASA (a list of the optional switches that can be used in the CHMTRN command is included in Appendix IV).

The CHMTRN program detected six errors, as shown below:

```
$ CHMTRN FGATB0
CHMTRN source file: LH_WORK:[LONG]FGATB0.SRC;6
1 transform found
Using CHMTRN image: LH_NEW:[AUX]CHMTRN.EXE;9

syntax error in line 549:
   KILL IF ATOM*1 IS NOT IN AN AROMATIC RING ?

syntax error in line 581:
   IF SPECIFIED*ATOM 1 IS NOT A FUSION ATOM GO ? TO CBETA

syntax error in line 588:
   SUBTRACT 20 IF ALPHA TO SPECIFIED ? ATOM 1 IS A  &
syntax error in line 591:
   CBETA ?          ENDDO

syntax error in line 594:
   CONDITIONS PH2:4 ?

Label CBETA undefined at end of table
Total errors found: 6
%SYSTEM-F-ABORT, abort
$
```

Careful examination of each offending line and a rereading of the sections of the CHMTRN Manual on RING, AROMATIC, GO TO, SPECIFIED*ATOM, and <CONDITIONS revealed two typographical errors and two attempts to put more than one <QLTARG into a single computer word. We hoped that the complaints about CBETA were caused by the illegal GO TO qualifier. The bad lines were easily rewritten as:

```
   KILL IF ATOM*1 IS NOT AN AROMATIC ATOM

   IF SPECIFIED*ATOM 1 IS NOT A FUSION ATOM THEN  &
     GO TO CBETA

   SUBTRACT 20 IF ALPHA TO SPECIFIED*ATOM 1 IS A  &
```

and

CONDITIONS **pH2:4**

respectively.

Running CHMTRN again gave no syntax errors. We responded "Yes" to the question about updating TRNSFM.DAT and we now had an FGATB0.BIN file we could try out with LHASA.

Second Try

Now we had a version of the transform that would generate precursors. For our first target structure, we selected the simplest, benzene. We ran LHASA, drew in a benzene ring using the first template, selected DEBUG on the strategy menu, then MANUAL FGA SELECTION on the Debug Menu, and answered the four questions as follows:

Object group origin? 3
Target types? 34
Path bonds? 2
Path change? 0

Predictably enough, benzene gave no increments or decrements. Our next three test structures were the three structures from the "Test Examples" section of the Reaction Summary Form (labeled I, II, and III in Figure 11.1 for convenience). Structure I, ortho-chlorotoluene, gave an increment of 0 when a C*SULFONATE was added para to the methyl group. This result was somewhat surprising in light of the qualifiers we had included about DONATING groups. We would have expected an increment of 10 for the methyl group GAMMA TO ATOM*1 and an additional increment of 5 for the chloride BETA TO ATOM*1. To find out why LHASA hadn't displayed a rating increment of 15, we ran the example again with the QUALIFIER TRACE, MECHANISM HANDLER, and FGA,FGR EXECUTIVES debug options selected. (Note that this repeat testing could also be done using LHINIT files as described in Appendix V.)

The top of the QUALIFIER TRACE section of the LHASA.LOG file generated is reproduced below:

```
EVLTRN
  MODE = 0
EVLINI entered. Group1: 00000000 Group2: 00000000 MODE:   0
EVLQLR

***    Table location:   546
   Operation: Rating
   ********** Transform 720 **********
      Proto-desulfonation of Aryl Sulfonic Acids
   Initial rating =  60

***    Table location:   548
   Operation: Kill if not
   Target: aromatic
   Locant: atom 1
   Locant atom set:    3
   Locant bond set:    2   3
```

```
Target atom set:   1  2  3  4  5  6
Target bond set:   1  2  3  4  5  6
Resultant set of atoms:   3
```

We took a look at the first transform qualifier to familiarize ourselves with the debug output. We remembered that our first qualifier had been KILL IF ATOM*1 IS NOT AN AROMATIC ATOM, and we saw that LHASA had crudely translated the qualifier back for us. The locant atom set contained the atom sequence number (3) of ATOM*1, as we had defined it in our MANUAL FGA SELECTION dialog. The locant bond set contained the bond numbers of the bonds (2, 3) attached to ATOM*1. The target atom set contained the atom numbers of all the AROMATIC atoms in the target structure (1-6), and the target bond set the bond numbers (1-6) of all the AROMATIC bonds. Since the qualifier asked about atoms rather than bonds, the resultant set of atoms contained the atoms that were on in both the locant atom set and the target atom set, namely 3.

To try to track down the problem with the DONATING qualifier, we looked ahead a few lines in LHASA.LOG and found the appropriate output:

```
***   Table location:   552
   Operation: Add 15 for each
   Target: donating group
   Locant: alpha to atom 1
   Locant atom set:   2   4
   Locant bond set:   1   4
   Target atom set: NONE
   Target bond set: NONE
   Resultant set of atoms: NONE
```

LHASA apparently found no donating groups (Target atom set = NONE) anywhere in the molecule. Perusal of the DONATING section in the CHMTRN Manual revealed that LHASA doesn't consider halides or alkyl groups to be donating groups, so we had to add a few lines of code to handle these cases:

```
...
... Increment for donating groups on the ring
...
      ADD 15 FOR EACH DONATING GROUP ON ALPHA TO ATOM*1
      ADD 15 FOR EACH HALIDE ON ALPHA TO ATOM*1
      ADD 15 FOR EACH ALKYL GROUP ON ALPHA TO ATOM*1
      ADD 10 IF GAMMA TO ATOM*1 ON*CURRENT*RING HAS A  &
        DONATING GROUP OR: HALIDE OR: ALKYL GROUP
      ADD  5 FOR EACH DONATING GROUP ON BETA TO ATOM*1  &
        ON*CURRENT*RING
      ADD  5 FOR EACH HALIDE ON BETA TO ATOM*1  &
        ON*CURRENT*RING
      ADD  5 FOR EACH ALKYL GROUP ON BETA TO ATOM*1  &
        ON*CURRENT*RING
```

Structure II gave the anticipated increment of -10, resulting from adding 10 for the para donating group and subtracting 10 for each meta nitro group. Structure III gave an overall increment of -5, which inspection of the debug file revealed resulted from adding 15 for an

ortho donating group and subtracting 20 for a ring fused meta/para. We expected from our CHMTRN code that LHASA would add 10 for non-hydrogen ortho substituents (ADD 10 FOR EACH NON HYDROGEN ON BETA TO ATOM*1 OFF*CURRENT*RING). The debug output for this qualifier

```
***   Table location:  565
   Operation: Add 10 for each
   Target: Non hydrogen(s)
   Locant: beta to atom 1 off current ring
   Locant atom set: NONE
   Locant bond set: NONE
   Target atom set:   1  3  4  5  10  13  14  16
   Target bond set:   3  4  11  14  15  17
   Resultant set of atoms: NONE
```

revealed that LHASA didn't find any atoms BETA TO ATOM*1 OFF*CURRENT*RING. We then realized that our inclusion of OFF*CURRENT*RING forced the program to grow outward only from ATOM*1, where it found nothing but an implicit hydrogen atom. We were also a bit suspicious of the interpretation LHASA gave to our NON HYDROGEN target specifier. In principle, the target atom set should have contained all the atoms in the structure that were not hydrogens, but obviously atom numbers 2, 6, 7, 8, and 9 were missing, among others. We were pretty puzzled for a while, but again the CHMTRN Manual (see HYDROGEN, HYDROGEN*ATOM) provided a solution. The problem was that HYDROGEN refers not to actual hydrogen atoms, but to atoms **bearing** hydrogens.

With these points in mind, we rewrote the qualifier as follows:

```
SAVE AS 1 ALPHA TO ATOM*1 ON*CURRENT*RING
ADD 10 FOR EACH NON HYDROGEN*ATOM ALPHA TO  &
  SAVED*ATOM 1 OFF*CURRENT*RING.
```

We now decided to get adventurous and try some structures not suggested by our collaborating chemist. First we tried an anthraquinone, which worked fine, correctly giving no increment for the fusion situation. Next we tried a variation on Structure II to make sure that we and LHASA agreed on the definition of a withdrawing group. As it turned out, it's lucky we did try this test, since we were no more in agreement about WITHDRAWING than we were about DONATING! Our test structure had one -SO3H group and one -COOH group in place of the two -NO2 groups in Structure II. LHASA gave the desulfonation transform an increment of 0, having added 10 for the amino group and subtracted 10 for only one of the withdrawing groups. Examination of the WITHDRAWING section in the CHMTRN Manual showed us that LHASA does not consider ACID to be a withdrawing group. Just to make sure that our hypothesis was correct, we tried one other experiment, substituting -COOMe for -COOH in our test structure. We were somewhat surprised to see that LHASA had again given the transform an increment of 0, evidently ignoring the carboxyl group again. This time the debug file and a more careful reading of the CHMTRN Manual provided the answer:

```
***   Table location:  569
   Operation: Subtract 10 for each
   Target: withdrawing group
```

```
    Locant: beta to atom 1 (on) current ring
    Locant atom set:    1   5
    Locant bond set:    5   6
    Target atom set:    1   8
    Target bond set:    9  10  11
    Resultant set of atoms:    1
 New rating =   60 Current increment =  -10
```

LHASA had found withdrawing groups (the C*SULFONATE and the ESTER, respectively) on atoms 1 and 8, not on atoms 1 and 5. Apparently, in order to use the WITHDRAWING specifier correctly, we needed to ask about bonds with withdrawing character rather than withdrawing groups (see CHMTRN Manual section on WITHDRAWING), and we rewrote the code as follows:

```
    SAVE AS 1 GAMMA TO ATOM*1 ON*CURRENT*RING
    SUBTRACT 15 IF THERE IS A WITHDRAWING BOND ON  &
      SAVED*ATOM 1 OFF*CURRENT*RING
    SUBTRACT 15 IF THERE IS AN ACID ON ALPHA TO  &
      SAVED*ATOM 1 OFF*CURRENT*RING
    SAVE AS 1 BETA TO ATOM*1 ON*CURRENT*RING
    SUBTRACT 5 FOR EACH WITHDRAWING BOND ON  &
      SAVED*ATOM 1 OFF*CURRENT*RING
    SUBTRACT 5 FOR EACH ACID ON ALPHA TO  &
      SAVED*ATOM 1 OFF*CURRENT*RING
```

Notice here that we made use of the fact that saving a locant in a SAVED*ATOM overwrites whatever was in that SAVED*ATOM set beforehand and also of the fact that there can be either one or more than one atom(bond) in a SAVED*ATOM(BOND) set.

Third Try

By now we had become confident enough in our CHMTRN-writing abilities that we thought we should remove the one nagging problem in the transform, the fact that we had really cheated in identifying anthraquinonish structures. As a final flourish, we decided to improve this section of code.

Here's the result:

```
...
...     Increment for ortho/meta fused systems, and
...     decrement for meta/para  fused systems, but
...     only if the fused systems are not anthraquinonish.
...
    FOR EACH CARBON BETA TO ATOM*1 ON*CURRENT*RING DO CBETA
      IF SPECIFIED*ATOM 1 IS NOT A FUSION ATOM  &
        THEN GO TO CBETA
      IF ALPHA TO SPECIFIED*ATOM 1 IS A KETONE
      BEGIN KETALPH1
        SAVE AS 1 THE PREVIOUS LOCANT
        IF ALPHA TO SAVED*ATOM 1 OFF*CURRENT*RING IS A  &
          NON FUSION ATOM OR: A NON AROMATIC ATOM  &
          THEN GO TO CBETA
```

```
            IF ALPHA TO SPECIFIED*ATOM 1 ON*CURRENT*RING IS  &
              THE SAME AS ALPHA TO ATOM*1 THEN  &
              SAVE AS 2 THE PREVIOUS LOCANT
            IF ALPHA TO SAVED*ATOM 2 IS A KETONE
           BEGIN KETALPH2
             SAVE AS 3 THE PREVIOUS LOCANT
             IF ALPHA TO SAVED*ATOM 3 OFF*CURRENT*RING  &
               IS A FUSION ATOM AND: AN AROMATIC ATOM  &
               THEN GO TO CBETA
           BLKEND KETALPH2
         BLKEND KETALPH1
          ADD 25 IF ALPHA TO SPECIFIED*ATOM 1 IS A FUSION  &
            ATOM AND:IF THE PREVIOUS LOCANT IS THE SAME AS  &
            ALPHA TO ATOM*1 ON*CURRENT*RING
          SUBTRACT 20 IF ALPHA TO SPECIFIED*ATOM 1 IS A  &
            FUSION ATOM AND:IF THE PREVIOUS LOCANT IS  &
            THE SAME AS GAMMA TO ATOM*1 ON*CURRENT*RING
CBETA  ENDDO
```

We then let our collaborating chemist take a quick look at the CHMTRN code. Calling upon his not inconsiderable experience with aromatic substitution reactions, he cited to us some evidence that halides and oxygen-based groups at the meta position(s) function not as donating groups, but rather as withdrawing groups. His suggestion was to remove the two lines:

```
ADD 5 FOR EACH DONATING GROUP ON BETA TO ATOM*1 &
  ON*CURRENT*RING
ADD 5 FOR EACH HALIDE ON BETA TO ATOM*1 &
  ON*CURRENT*RING
```

which we promptly did.

Further testing by our collaborating chemist did not reveal any immediate problems, and he pronounced the transform ready to add to the LHASA knowledge base. The final version of the transform is shown below in Figure 11.3.

```
TRANSFORM 720
NAME Proto-desulfonation of Aryl Sulfonic Acids

...Sulfonation and Related Reactions, E.E. Gilbert,
...   Interscience, 1965, pp. 425-442.
...Mechanistic Aspects in Aromatic Sulfonation and
...   Desulfonation, H. Cerfontain, Interscience,
...   1965, pp. 185-195.
...The Organic Chemistry of Sulfur, C.M. Suter,
...   J. Wiley and Sons, 1944, pp. 387-394.
...Methoden der Organischen Chemie, v. 9, 1955, pp. 535-539.
...
     TYPICAL*YIELD                GOOD
     RELIABILITY                   FAIR
     REPUTATION                   GOOD
     HOMOSELECTIVITY              FAIR
```

```
      HETEROSELECTIVITY              FAIR
      ORIENTATIONAL*SELECTIVITY      EXCELLENT
      CONDITION*FLEXIBILITY          FAIR
      THERMODYNAMICS                 GOOD
...
... H        SO3H
...  |        |
... AR => AR
...
      ...Authors: Tony Baxter (ICI) and Alan Long (Harvard)
      ...Completed: August 1987
       ...PATH CHANGE 0
       GROUP*1 IS C*SULFONATE
       ...
       KILL IF ATOM*1 IS NOT AN AROMATIC ATOM
       KILL IF THERE IS NOT A HYDROGEN ON ATOM*1
       DESIGNATE THE RING CONTAINING ATOM*1 AS THE CURRENT*RING
...
...    Increment for donating groups on the ring
...
       ADD 15 FOR EACH DONATING GROUP ON ALPHA TO ATOM*1
       ADD 15 FOR EACH HALIDE ON ALPHA TO ATOM*1
       ADD 15 FOR EACH ALKYL GROUP ON ALPHA TO ATOM*1
       ADD 10 IF GAMMA TO ATOM*1 ON*CURRENT*RING HAS A  &
         DONATING GROUP OR: HALIDE OR: ALKYL GROUP
       ADD  5 FOR EACH ALKYL GROUP ON BETA TO ATOM*1  &
         ON*CURRENT*RING
...
...    Decrement for withdrawing groups on the ring
...
       SAVE AS 1 GAMMA TO ATOM*1 ON*CURRENT*RING
       SUBTRACT 15 IF THERE IS A WITHDRAWING BOND ON  &
        SAVED*ATOM 1 OFF*CURRENT*RING
       SUBTRACT 15 IF THERE IS AN ACID ON ALPHA TO  &
        SAVED*ATOM 1 OFF*CURRENT*RING
       SAVE AS 1 BETA TO ATOM*1 ON*CURRENT*RING
       SUBTRACT 5 FOR EACH WITHDRAWING BOND ON  &
        SAVED*ATOM 1 OFF*CURRENT*RING
       SUBTRACT 5 FOR EACH ACID ON ALPHA TO SAVED*ATOM 1  &
         OFF*CURRENT*RING
...
...    Increment for substituents in the ortho positions for
...      relief of steric strain
...
       SAVE AS 1 ALPHA TO ATOM*1 ON*CURRENT*RING
       ADD 10 FOR EACH NON HYDROGEN*ATOM ALPHA TO  &
        SAVED*ATOM 1 OFF*CURRENT*RING
...
...    Increment for ortho/meta fused systems, and
...    decrement for meta/para  fused systems, but
```

```
...     only if the fused systems are not anthraquinonish.
...
        FOR EACH CARBON BETA TO ATOM*1 ON*CURRENT*RING DO CBETA
          IF SPECIFIED*ATOM 1 IS NOT A FUSION ATOM  &
            THEN GO TO CBETA
          IF ALPHA TO SPECIFIED*ATOM 1 IS A KETONE
          BEGIN KETALPH1
            SAVE AS 1 THE PREVIOUS LOCANT
            IF ALPHA TO SAVED*ATOM 1 OFF*CURRENT*RING IS A  &
              NON FUSION ATOM OR: A NON AROMATIC ATOM  &
              THEN EXIT KETALPH1
            IF ALPHA TO SPECIFIED*ATOM 1 ON*CURRENT*RING IS  &
              THE SAME AS ALPHA TO ATOM*1 THEN  &
              SAVE AS 2 THE PREVIOUS LOCANT
            IF ALPHA TO SAVED*ATOM 2 IS A KETONE
           BEGIN KETALPH2
             SAVE AS 3 THE PREVIOUS LOCANT
             IF ALPHA TO SAVED*ATOM 3 OFF*CURRENT*RING  &
                IS A FUSION ATOM AND: AN AROMATIC ATOM  &
                THEN GO TO CBETA
           BLKEND KETALPH2
         BLKEND KETALPH1
          ADD 25 IF ALPHA TO SPECIFIED*ATOM 1 IS A FUSION  &
            ATOM AND:IF THE PREVIOUS LOCANT IS THE SAME AS  &
            ALPHA TO ATOM*1 ON*CURRENT*RING
          SUBTRACT 20 IF ALPHA TO SPECIFIED*ATOM 1 IS A  &
            FUSION ATOM AND:IF THE PREVIOUS LOCANT IS THE  &
            SAME AS GAMMA TO ATOM*1 ON*CURRENT*RING
CBETA   ENDDO
        CONDITIONS pH<1
        CONDITIONS pH:1
        CONDITIONS pH2:4
        CONDITIONS pH4:6
        ....
          ATTACH A C*SULFONATE TO ATOM*1
        ....
```

Figure 11.3 Proto-Desulfonation Transform - Final Version

Appendix I - LHASA Superatom Types

Superatom Types for Use in 2-D Transform Patterns

-H	-H
-HELIUM	-HELIUM
-LI	-LI
-BE	-BE
-B	-B
-C	-C
-N	-N
-O	-O
-F	-F
-NEON	-NEON
-NA	-NA
-MG	-MG
-AL	-AL
-SI	-SI
-P	-P
-S	-S
-CL	-CL
-ARGON	-ARGON
-K	-K
-CA	-CA
-SC	-SC
-TI	-TI
-V	-V
-CR	-CR
-MN	-MN
-FE	-FE
-CO	-CO
-NI	-NI
-CU	-CU
-ZN	-ZN
-GA	-GA
-GE	-GE
-AS	-AS
-SE	-SE
-BR	-BR
-KRYP	-KRYP
-RB	-RB
-STR	-STR
-Y	-Y
-ZR	-ZR
-NB	-NB
-MO	-MO
-TC	-TC
-RU	-RU

-RH	-RH
-PD	-PD
-AG	-AG
-CD	-CD
-IN	-IN
-SN	-SN
-SB	-SB
-TE	-TE
-I	-I
-XE	-XE
-Z	-Z
-PLUS	-PLUS
-X	-F,-CL,-BR,-I
=N	=N
=NH	=N-H
=NR	=N-C
#N	#N
/N	-N
%N	%N
%NN	%N%N
%NO	%N%O
%NR	%N%C
-NCO	-N=C=O
-NR2	-N(-C)-C
-NHR	-N(-H)-C
-NH2	-N(-H)-H
-NH	-N-H
-NRN	-N-N
=N2	=N=N
-N3	-N=N2
-NO	-N=O
-NX	-N-X
=NNH2	=N-N
=NOH	=N-O
-NCR2	-N=C
-NC	-N#C
-N2R	-N=N
-NHOH	-N(-H)-O
-NROH	-N(-C)-O
-NOH	-N-O
-NO2	-N(=O)-O
-NCONH	-N-C(=O)-N-H
-NHCON	-N(-H)-C(=O)-N
-NRCON	-N(-C)-C(=O)-NR2
-OCON	-O-C(=O)-N
-SIR3	-SI(-C)(-C)-C
#O	#O
=O	=O
/O	-O
%O	%O

%OR	%O%C
%ON	%O%N
-OSIR3	-O-SIR3
-OP	-O-P
-OR	-O-C
-OH	-O-H
-OOH	-O-OH
-OOR	-O-OR
-ONR2	-O-N
-NCOOR	-N-C(=O)-O-C,-N-C(=O)-OR,-N(-C)-C(=O)-OR
-OCOOR	-O-C(=O)-O-C,-O-C(=O)-OR
-OSO2R	-O-S(=O)(=O)-C
=C	=C
=CH2	=C(-H)-H
#C	#C
%C	%C
-COR	-C(=O)-C
-CHO	-C(=O)-H
-COOH	-C(=O)-OH
-COOR	-C(=O)-OR
-CONH2	-C(=O)-NH2
-CONHR	-C(=O)-NHR
-CONR2	-C(=O)-NR2
-COX	-C(=O)-X
-CNR	-C=N
-CN	-C#N
-CX2	-C(-X)-X
-CX3	-C(-X)(-X)-X
-COHOR	-C(-OH)-OR
-COROR	-C(-OR)-OR
-CH3	-C(-H)(-H)-H
-CH2OH	-C(-H)(-H)-OH
-PH	-C%C
-AR	-C%C,-C%N,-C%S,-C%O
=S	=S
/S	-S
%S	%S
%SR	%S%C
-SH	-S-H
-SR	-S-C
-SSR	-S-SR
-SO	-S=O
-SO2	-S(=O)=O
-SO3	-S(=O)(=O)-O
-SOR	-S(=O)-C
-SO2R	-S(=O)(=O)-C
-SO3R	-S(=O)(=O)-OR
-SCN	-S-CN
-COSR	-C(=O)-SR
-SER	-SE-C

-OCOR	-O-C(=O)-C,-O-C(=O)-H,-O-C=O,-O-COR
-NCOR	-N-C(=O)-C,-N-C(=O)-H,-N(-C)-C=O,-N-COR
-COOCOR	-C(=O)-OCOR
-W	-COR,-CHO,-COOR,-CONH2,-CONHR,-CONR2,-COX, -COSR,-C=N,-C=NOH,-CN,-NO2,-SOR,-SO2R,-SO3R,CX3, -C=C-W,-COOCOR
-CO2	-C(=O)=O

Appendix II - LHASA Functional Groups

LHASA Explicit Functional Groups

ACETAL	C*SULFONATE	HALOAMINE	OLEFIN
ACETYLENE	CARBAMATE*C	HALOHYDRIN	OXIME
ACID	CARBAMATE*H	HEMIACETAL	PEROXIDE
ACID*HALIDE	CARBONIUM	HYDRAZONE	PHOSPHINE
ALCOHOL	CHLORIDE	HYDROXYLAMINE	PHOSPHONATE
ALDEHYDE	DIAZO	IMINE	SELENIDE
ALLENE	DISULFIDE	IODIDE	SILYLENOLETHER
AMIDE*1	DITHIOACETAL	ISOCYANATE	SULFIDE
AMIDE*2	DITHIOKETAL	ISOCYANIDE	SULFONE
AMIDE*3	ENAMINE	KETONE	SULFOXIDE
AMIDZ	ENOL*ETHER	N*CARBAMATE	THIOCYANATE
AMINE*1	EPISULFIDE	N*UREA*C	THIOESTER
AMINE*2	EPOXIDE	N*UREA*H	THIOL
AMINE*3	ESTER	NITRILE	TRIALKYLSILOXY
ANHYDRIDE	ESTERX	NITRO	TRIALKYLSILYL
AZIDE	ETHER	NITROSO	TRIHALIDE
AZIRIDINE	FLUORIDE	O*CARBAMATE	VIC*DIHALIDE
AZO	GEM*DIHALIDE	O*CARBONATE	VINYLD
BROMIDE	GLYCOL	O*SULFONATE	VINYLSILANE
			VINYLW

LHASA Generic Functional Groups

AMIDE:	AMIDE*1	
	AMIDE*2	
	AMIDE*3	
AMINE:	AMINE*1	
	AMINE*2	
	AMINE*3	
CARBONYL:	ALDEHYDE	KETONE
CARBOXYL:	ACID	AMIDE*3
	ACID*HALIDE	ANHYDRIDE
	AMIDE*1	ESTER
	AMIDE*2	THIOESTER
DONATING:	ACETAL	N*CARBAMATE
	ALCOHOL	N*UREA*H
	AMIDZ	N*UREA*C
	AMINE*1	O*CARBAMATE
	AMINE*2	O*CARBONATE

	AMINE*3	PHOSPHINE
	ESTERX	SELENIDE
	ETHER	SULFIDE
	HALOAMINE	THIOL
	HEMIACETAL	TRIALKYLSILOXY
	HYDROXYLAMINE	VINYLD
HALIDE:	BROMIDE	GEM*DIHALIDE
	CHLORIDE	IODIDE
	FLUORIDE	TRIHALIDE
LEAVING:	ACETAL	IODIDE
	ALCOHOL	N*CARBAMATE
	AMIDZ	N*UREA*C
	AZIRIDINE	N*UREA*H
	BROMIDE	O*CARBAMATE
	C*SULFONATE	O*CARBONATE
	CHLORIDE	O*SULFONATE
	DITHIOACETAL	PHOSPHONATE
	DITHIOKETAL	SELENIDE
	EPISULFIDE	SULFIDE
	EPOXIDE	SULFONE
	ESTERX	THIOCYANATE
	ETHER	THIOL
	FLUORIDE	TRIALKYLSILYLOXY
	GEM*DIHALIDE	TRIHALIDE
GOOD*LEAVING:	AZIRIDINE	GEM*DIHALIDE
	BROMIDE	IODIDE
	CHLORIDE	O*SULFONATE
	EPISULFIDE	PHOSPHONATE
	EPOXIDE	TRIHALIDE
NONXWITHDRAWING:	C*SULFONATE	SULFONE
	NITRO	SULFOXIDE
XWITHDRAWING:	ACID*HALIDE	IMINE
	ALDEHYDE	KETONE
	AMIDE*	NITRILE
	AMIDE*2	OXIME
	AMIDE*3	THIOESTER
	ANHYDRIDE	TRIHALIDE
	CARBONIUM	VINYLW
	ESTER	

Note that British spellings of functional groups (e.g. SULPHIDE) are equally acceptable.

Appendix III - Reaction Summary Form

Reaction Summary Form

Name **Date**

Lab.No. **Phone**

Reaction Name (e.g. Michael Addition)

Short Description (complete if name not satisfactory)

Stereospecific? Yes No **Regiospecific?** Yes No

References (include a review if possible)

Company Secret? Yes No

Not all of the following sections will be relevant in all cases.
Please fill out all you can, attaching further sheets if necessary.

Generalized Reaction Description

Please draw in a retrosynthetic sense the smallest substructure that is essential, or unavoidable, for the reaction. Complete all valencies with numbered groups. The following abbreviations may be useful: W = withdrawing, D = donating, X = leaving group. Please number the product, starting with any convenient atom as number 1, numbering essential or unavoidable appendages as you come to them. Then number the starting material(s), using the product numbering for corresponding atoms. Finally, for pattern chemistry transforms, write out a draft pattern.

Structural Requirements of Starting Materials

How would various groups near or adjacent to the reaction site help or hinder the reaction? Would some combinations completely prevent the reaction from occurring? Consider withdrawing, donating, and leaving groups as well as steric bulk. If possible, tabulate and include yield ranges.

Consider bond by bond whether any bond can or cannot be cyclic, acyclic, or aromatic. If a bond is a ring bond, does this factor have an effect on the yield? Refer to bonds by atom numbers from the previous page.

Reaction Conditions

A. Reagents, solvents, pH, temperature, etc.

<.figure 10>

B. Work-up

<.figure 10>

Initial Rating for Transform

Note: Refer to Chapter 9 of Transform Writers' Guide for definitions of the following scales:

	Bad	Poor	Fair	Good	V.G.	Excellent
Typical yield	[]	[]	[]	[]	[]	[]
Reliability	[]	[]	[]	[]	[]	[]
Reputation	[]	[]	[]	[]	[]	[]
Homoselectivity	[]	[]	[]	[]	[]	[]
Heteroselectivity	[]	[]	[]	[]	[]	[]
Orientational selectivity	[]	[]	[]	[]	[]	[]
Condition flexibility	[]	[]	[]	[]	[]	[]
Thermodynamics	[]	[]	[]	[]	[]	[]

Test Examples

Please provide at least three good test cases, with yields if possible. These may be to either succeed or fail.

Appendix IV - CHMTRN Compiler Commands

A command procedure (LHA:CHMTRN.COM) is used to translate transform source files into binary versions suitable for use with the LHASA program. The procedure is run using the DCL symbol of the same name, as in:

```
$ CHMTRN PCHEM1 PATRAN.
```

CHMTRN can also be run in batch mode via the command BCHMTRN, which takes all of the same arguments as the CHMTRN command.

The first parameter is the name of the input file. Subsequent parameters may be any of the following:

DEBUG	Prints out lots of information about parsing.
DEBUG=linenum	Begins printout at specified line of input file.
LIST	Creates a listing file with same name as input that includes CHMTRN information and, if PATRAN is run, a second listing file including PATRAN information.
LIST=listfile	Create a CHMTRN listing file called <listfile>. The PATRAN listing file name cannot be changed.
PATRAN	Runs PATRAN after CHMTRN for files containing 1-D patterns.
PLIST	Creates a listing file with same name as input that includes PARSE information (2-D patterns).
PLIST=listfile	Create a PARSE listing file called <listfile>.
SAVE_CSR	Saves the subroutines used in the source file in a separate file called <name>.CSR.
TLIST	Has TABLE_FINDER create a listing file with same name as input that includes information on what transforms will be selected as candidates for each step of each TC.

The only parameters that a transform writer will be likely to use are LIST and PATRAN. LIST generates a line-numbered listing of the CHMTRN source code, less useful now that the qualifier trace debug output from LHASA contains textual translations of transform qualifiers. PATRAN runs the PATRAN program that processes one-dimensional patterns in PCHEMx tables and generates PATRNx.BIN files for use by the 1-D-pattern-matching modules in LHASA.

More information on these and other offline auxiliary programs supplied with LHASA is available in LHL:LHASA_AUX.HLB, a local help library that can be accessed via the DCL symbol HELPAUX.

Appendix V - Batch-mode Replay Files for LHASA

It is often convenient when testing a new transform to create a replay file that contains a summary of the button hits made in a preliminary run. This replay file can be used to run LHASA automatically in subsequent trials. To make a replay file, run LHASA once with a file called LHINIT.DAT in your disk area containing the following lines:

```
<SPACE>$LHASA
<TAB>REPLAY_FILE = 'REPLAY',
<TAB>GRAPHICAL_INPUT = 'SAVE',
<TAB>BATCH_MODE = .TRUE.,
<SPACE>$END
```

In this preliminary run, you may either draw in your target structure from scratch or you may access a previously-saved structure via the TRACK button.

For subsequent tests in automatic (replay) mode, run the program with a new LHINIT.DAT file containing:

```
<SPACE>$LHASA
<TAB>REPLAY_FILE = 'REPLAY',
<TAB>GRAPHICAL_INPUT = 'REPLAY',
<TAB>BATCH_MODE = .TRUE.,
<TAB>CONTIN_YESNO = 1,
<TAB>STER_YESNO = -2,
<SPACE>$END
```

Note that this procedure will not be completely automatic for runs involving MANUAL FGI SELECTION or MANUAL FGA SELECTION, since the responses to the dialogs initiated by these two modes are not recorded in the replay file that you create. You will have to type in the appropriate responses yourself each time.

As described in detail in the LHASA Users' Guide, a number of other variables can be initialized via LHINIT.DAT files. Among the most useful of these is the DBGTEXT array of debugging keywords, since it shortcuts the process of selecting the corresponding buttons on the DEBUG menu in LHASA. For example, one could turn on four particularly useful debugging buttons with the following entry in LHINIT.DAT:

```
<TAB>DBGTEXT = 'QUALIFIER TRACE',
<TAB><TAB>'MECHANISM HANDLER',
<TAB><TAB>'CHEMISTRY EXECUTIVES',
<TAB><TAB>'FGI EXECUTIVES'
```

Appendix V1 – New 1D Patterns

Philip Judson
6th July 2017

Introduction

An extension has been added to the 1-D pattern language, PATRAN, so that the precursors to a reaction can be included in the pattern, to support pattern-based generation of forward reactions from 1-D retrosynthetic descriptions. 1-D patterns are easier to write than 2-D patterns and they support all the atom and bond properties defined in PATRAN, many of which cannot be used in 2-D patterns.

This appendix describes the extension.

Adding a New 1-D Pattern to a CHMTRN Transform

Extended 1-D patterns are included in transforms between the conventional 1-D patterns and 2-D patterns, and are preceded by a "NEW*1D*PATTERN statement. For example:

```
1D*PATTERN
C[ARYL=YES])-C#C[HETS=0]
NEW*1D*PATTERN
C[ARYL=YES])-C#C[HETS=0] => C^1[ARYL=YES])-I + C^2[HS=1]#C^3[HETS=0]
2D*PATTERN
                        I    H
                        |    |
      AR-C#C-C => AR  C#C-C
END*PATTERNS
```

Writing a new 1-D pattern

The first part of a new 1-D pattern is the same as a conventional 1-D pattern and if both a 1-D and a new 1-D pattern are used in the same transform it must be identical to it. It is followed by a retrosynthetic arrow represented by an equals sign and a greater than sign and then the 1-D pattern(s) for the precursor(s) separated by plus signs:

target_pattern => precursor_1_pattern + precursor_2_pattern + ...

Each of the target and precursor fragments is described in the same way as fragments in conventional 1-D patterns but, in addition, the mapping of each precursor atom to each atom also present in the target is shown by a carat (^) character and the number of the corresponding atom in the precursor pattern. For example the formation of a pyrrole from a 1,4-diketone and primary amine might be represented by:

N(-C)%C(-C)%C%C%C(-C)%@1 => N^1-C^2 + O=C^3(-C^4)-C^5-C^6-C^7(-C^8)=O

The ordering of atoms in the precursor patterns need not necessarily be the same as in the target pattern. In this case, for example, the following pattern would be equally valid:

N(-C)%C(-C)%C%C%C(-C)%@1 => O=C^3(-C^4)-C^5-C^6-C^7(-C^8)=O + N^1-C^2

In a real transform, attributes would probably be included for some atoms and bonds (e.g. [HS=2]) but they have been omitted from these examples for simplicity.

If new atoms are present in the precursors they are numbered automatically from left to right, continuing from the highest numbered atom in the target. In this case the two oxygen atoms in the precursor fragment are thus ATOM*9 and ATOM*10. New bonds (as distinct from bonds that have only undergone a change in bond order) are similarly numbered from left to right. The two double bonds in the above examples are thus BOND*9 and BOND*10.

Simple logic dictates that, since all atoms and bonds in a target are numbered automatically and a pattern is always a list of alternating atoms and bonds starting with an atom, the number of a bond in the target is always the same as the number of the atom to its left. Note that this is not the case in the precursors. If a bond is to be referred to in CHMTRN statements care should be taken to find its bond number by counting the bonds in the target and new bonds in the precursors, not the atoms. For example in the pattern above, the atom to the left of the second double bond in the precursor is ATOM*8 but the bond is BOND*10.

As with conventional 1-D and 2-D patterns, multiple new 1-D patterns are allowed in a transform provided that atoms and bonds with the same numbers in different patterns (including numbers generated automatically) correspond to each other. For example using the pair of patterns

C-N-C[HS=2] => C^1-BR,CL,I + N^2-C^3[HS=2]
C-N-C=O => C^1-BR,CL,I + N^2-C^3=O^4

is not valid because in the first pattern the halogen atom would be numbered 4 automatically whereas in the second pattern atom number 4 is the carbonyl oxygen atom and the halogen atom would be numbered 5.

Use of atom and bond lists and generic atoms

Atom and bond lists, and generic atoms and bonds (X and &) are permitted in new 1-D patterns, as they are in conventional patterns. Lists for an atom that is present on both sides of a transform arrow must be the same on both sides. Direct correspondence of each specific atom type in the list is assumed. For example,

C-O,S-C-C-C(=O)-O[HS=1] => **C^1-O,S^2-C^3**-C^4-C^5(=O^6)-BR

is equivalent to the pair of patterns

C-O-C-C-C(=O)-O[HS=1] => **C^1-O^2-C^3**-C^4-C^5(=O^6)-BR
C-S-C-C-C(=O)-O[HS=1] => **C^1-S^2-C^3**-C^4-C^5(=O^6)-BR

The other pair of patterns that could be imagined, in which an oxygen would turn into a sulphur atom and *vice versa*, do not make chemical sense and they are ignored automatically.

Unlike changes of type for the same atom, changes of bond order can be valid. For example

C-C => C^1=C^2

represents the reduction of a double bond and the following pattern covers reduction of both double and triple bonds by the same transform

C-C => C^1=,#C^2

However, if there are lists for a bond on both sides of the arrow the lists must be the same and they are interpreted on a one-to-one basis in the same way as atom lists. For example

C=,#C-C-O[HS=1] => C^1=,#C^2-C^3-BR

is eqivalent to the pair of patterns

C=C-C-O[HS=1] => C^1=C^2-C^3-BR
C#C-C-O[HS=1] => C^1#C^2-C^3-BR

Patterns containing a list of types of bond in the target structure but not in the precursor are not supported (and, as mentioned above, they are not allowed for lists of atom types).

An important note on standard 1-D Patterns

It is strongly recommended that where multiple conventional 1-D patterns are used in a transform the atom counts are the same in all of them, even if no new 1-D patterns are used in the transform, to avoid ambiguity about the numbering of new atoms in precursors.

Writing transforms for forward chemistry

New 1-D patterns support the writing of transforms describing forward chemistry (as distinct from retrosynthetic reactions).

The transform must contain a FORWARD* REACTION statement after the references:

END*REFERENCES
...
FORWARD*REACTION
...

The only difference in a new 1-D pattern describing forward chemistry, in addition to the reversal order of products and reactants, is that a reaction arrow, –>, is used in place of a retrosynthetic arrow, =>.

For example, the retrosynthetic and synthetic (forward chemistry) patterns for the formation of a pyrrole from a 1,4-diketone and primary amine might be:

N(-C)%C(-C)%C%C%C(-C)%@1 => N^1-C^2 + O=C^3(-C^4)-C^5-C^6-C^7(-C^8)=O

and

N-C + O=C(-C)-C-C-C(-C)=O –> N^1(-C^2)%C^4(-C^5)%C^6%C^7%C^8(-C^9)%@1

Reformatted for Word
and Appendix VI added
27th July 2017

Additions made to Appendix V1
10th August 2017
and 21st December 2017
Philip Judson